

Análisis FAPI 2.0 para Finanzas Abiertas (Chile) y soporte en Keycloak — v3

Versión 3 del documento. Mantiene los 18 requisitos originales (secciones 1-18) y **expande** los 26 aspectos previamente agrupados en la sección 19 (resumen) como **puntos independientes** (secciones 19-44), cada uno con la misma estructura analítica del resto del documento.

Documento técnico que evalúa los **44 requisitos de seguridad** exigibles al *Authorization Server* (Identity Server) bajo FAPI 2.0, en el contexto del **Sistema de Finanzas Abiertas (SFA)** de la **CMF de Chile** (NCG N.º 514 y su normativa técnica complementaria, basada en el perfil **FAPI 2.0 Security Profile + Message Signing** de la OpenID Foundation).

Para cada ítem se entrega:

1. Contexto técnico (definición, diagrama, ejemplos).
2. Obligatoriedad en FAPI 2.0 / SFA Chile.
3. Soporte nativo en **Keycloak ≥ 26.x**.
4. Recomendaciones de implementación si no está soportado.

Histórico de versiones:

- **v1** — 18 requisitos + sección 19 resumen + apéndices (PDF de 26 páginas).
- **v2** — PDF distribuible reducido (secciones 1-18 únicamente).
- **v3** (*este documento*) — secciones 19-44 expandidas como puntos independientes.

Tabla de Acrónimos

Sigla	Significado
FAPI	Financial-grade API (perfil de seguridad OAuth/OIDC de la OpenID Foundation)
SFA	Sistema de Finanzas Abiertas (Chile)
CMF	Comisión para el Mercado Financiero
SSA	Software Statement Assertion (RFC 7591)
DCR	Dynamic Client Registration (RFC 7591/7592)
PAR	Pushed Authorization Requests (RFC 9126)
JAR	JWT-Secured Authorization Request (RFC 9101)
JARM	JWT Secured Authorization Response Mode (OIDF)
PKCE	Proof Key for Code Exchange (RFC 7636)
DPoP	Demonstrating Proof of Possession (RFC 9449)
mTLS	Mutual TLS for OAuth (RFC 8705)
RAR	Rich Authorization Requests (RFC 9396)
OCSP	Online Certificate Status Protocol (RFC 6960)
SPI	Service Provider Interface (mecanismo de extensión de Keycloak)

Resumen ejecutivo – Matriz de cumplimiento

#	Requisito	¿Obligatorio FAPI 2.0?	Soporte Keycloak 26.x
1	Compatible con FAPI 2.0	Sí	✓ Parcial (Security Profile, Message Signing en <i>preview</i>)
2	mTLS 1.3	Sí (mTLS), TLS 1.3 recomendado	✓ Soportado (depende del <i>reverse-proxy</i>)
3	ES256 / PS256	Sí	✓ Soportado
4	Rotación de Refresh Tokens	Sí (o sender-constrained)	✓ Soportado
5	Deshabilitar flujos legacy	Sí	✓ Soportado vía <i>Client Policies</i>
6	Sender-Constrained Tokens	Sí (mTLS o DPOP)	✓ Soportado
7	DPoP	Alternativa obligatoria	✓ GA desde 26.x
8	DCR	Sí (Open Finance)	✓ Parcial (sin validación de SSA out-of-the-box)
9	Consumo de SSA	Sí (SFA Chile)	✗ Requiere SPI
10	JARM	Sí en <i>Message Signing</i>	✓ Soportado
11	PAR obligatorio	Sí	✓ Soportado
12	PKCE obligatorio	Sí (S256)	✓ Soportado
13	OCSP	Sí (cadena PKI)	⚠ Parcial
14	OCSP Stapling	Recomendado	⚠ Vía reverse-proxy
15	Consentimiento granular	Sí (SFA)	⚠ Parcial
16	RAR (RFC 9396)	Opcional en FAPI 2.0, exigido por SFA	⚠ Parcial (claim <code>authorization_details</code> configurable)
17	Mapeo de scopes ↔ recursos legales	Sí (SFA)	✗ Requiere extensión
18	Logs criptográficos no-repudiables	Sí (SFA, trazabilidad)	✗ Requiere extensión
19	Identificación del Issuer en respuestas (RFC 9207)	Sí	✓ Soportado
20	JAR / Request Object firmado (RFC 9101)	Sí (Msg Signing) / Recomendado (Sec Profile)	✓ Soportado
21	Client Authentication (<code>private_key_jwt</code> / <code>tls_client_auth</code>)	Sí	✓ Soportado
22	JWKS rotativo del cliente (<code>jwtks_uri</code>)	Sí	✓ Soportado
23	<code>aud</code> / <code>exp</code> estrictos en <code>client_assertion</code>	Sí	✓ Soportado
24	CIBA Backchannel	Recomendado / Obligatorio en casos <i>decoupled</i> SFA	✓ Soportado
25	CIBA Delivery Modes (<code>ping</code> / <code>poll</code> / <code>push</code>)	Sí (SFA: <code>ping</code> o <code>poll</code>)	✓ Parcial (<code>push</code> limitado)

#	Requisito	¿Obligatorio FAPI 2.0?	Soporte Keycloak 26.x
26	ACR / AMR — niveles de autenticación	Sí (SCA)	✓ Soportado
27	WebAuthn / FIDO2 para SCA	Recomendado	✓ Soportado
28	RP-Initiated Logout (id_token_hint)	Sí	✓ Soportado
29	Back-Channel Logout (OIDC BCL 1.0)	Sí (revocación de sesión)	✓ Soportado
30	Algoritmos JWE para claims sensibles	Opcional / Recomendado para PII	✓ Soportado
31	Cripto-agilidad y <i>post-quantum readiness</i>	Roadmap	⚠ Parcial
32	Rate limiting / anti-bruteforce en endpoints OAuth	Sí	⚠ Parcial (login user); requiere gateway
33	Resiliencia / HA del AS	Sí (SLA)	✓ Soportado (Infinispan + DB HA)
34	Time synchronization (NTP / NTS)	Implícito	⚠ Depende del SO
35	Inventario de eventos + retención (5-10 años)	Sí	⚠ Parcial; requiere storage WORM externo
36	Conformance OI DF + Pen-testing recurrentes	Sí (CMF)	N/A (proceso)
37	Mapeo de acr canónico SFA (urn:cmf:cl:sfa: ...)	Sí (cuando CMF publique)	✓ Soportado vía mapping
38	Notificación de eventos al cliente (SET / RFC 8417)	Sí (revocación de consentimiento)	✗ Requiere SPI + dispatcher
39	sub pseudonymous / pairwise / no-PII	Sí	✓ Soportado
40	Exact-match de redirect_uri	Sí	✓ Soportado (bloquear * por policy)
41	Prohibición de request_uri pre-registrado	Sí	✓ Soportado por profile
42	Consent API formal (CRUD de consent_id)	Sí	✗ Servicio externo
43	Aislamiento por tenant / realm	Sí	✓ Soportado
44	Política de claves: separación de roles + HSM	Sí	⚠ Parcial (HSM vía PKCS#11)

Leyenda: ✓ soporte nativo, ⚠ soporte parcial, ✗ no soportado nativamente.

1. Compatible con FAPI 2.0

1.1 Contexto técnico

FAPI 2.0 es la evolución de FAPI 1.0 publicada por la **OpenID Foundation Financial-grade API Working Group**. Está compuesto por dos perfiles formales:

- **FAPI 2.0 Security Profile** (ID-2 / Final): conjunto **base** de controles que el AS y el cliente deben cumplir.

- **FAPI 2.0 Message Signing**: capa adicional **sobre** el Security Profile que exige firma JWS de mensajes (request object + JARM) para escenarios de no-repudio (típicamente *write/payment*).

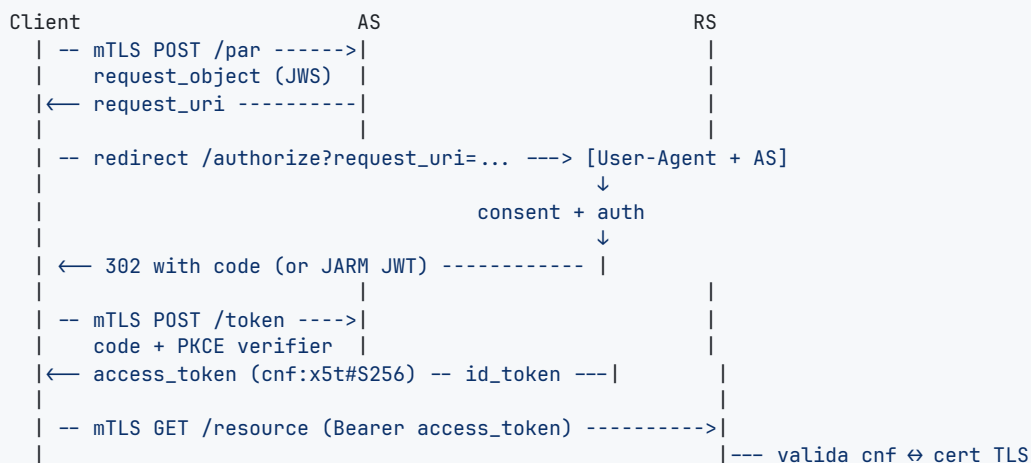
Se construyó como simplificación del FAPI 1.0 Advanced eliminando ambigüedades, removiendo `response_type=code id_token`, exigiendo PAR siempre y eliminando `s_hash`.

Modelo de amenazas cubierto

FAPI 2.0 está formalmente verificado contra el **Attacker Model** definido por OIDF, cubriendo:

- Atacante de red (incluido TLS-MITM con certificado válido para otra entidad).
- Atacante con control de un endpoint del cliente.
- Atacante con acceso al *user agent*.
- Cross-site request injection.
- Mix-up attacks (mediante `iss` en la respuesta — RFC 9207).

Diagrama (flujo "code with PAR + mTLS")



1.2 ¿Obligatorio?

Sí. La NCG 514 de la CMF y la "Norma Técnica de Seguridad del SFA" establecen que la autenticación y autorización **debe** seguir FAPI 2.0 Security Profile, y en operaciones con efectos financieros (movimiento de fondos, iniciación de pagos) **FAPI 2.0 Message Signing**.

1.3 Soporte Keycloak

✅ **Parcial.** Keycloak incorpora desde la versión 22.x un *Client Policy Profile* llamado `fapi-2-security-profile` y `fapi-2-message-signing`. Activarlo:

```
kc.sh start --features=fapi-2,par,dpop,token-exchange
```

Y en la realm:

```
Realm Settings → Client Policies → Profiles → fapi-2-security-profile
                  → Policies → "FAPI 2.0 clients" (matcher por client-attr)
```

Limitaciones conocidas (a la fecha):

- El profile no fuerza automáticamente `iss` en la respuesta de autorización (RFC 9207); está, pero hay que validar la versión.
- El profile de Message Signing es Tech-Preview en algunas releases LTS, GA en 26.x.

1.4 Recomendaciones

- Mantener Keycloak en **26.x LTS** o superior.
- Validar la conformidad ejecutando la **OpenID Foundation Conformance Suite** (`fapi2-security-profile-id2-test-plan`).
- Documentar internamente el *gap* respecto al perfil chileno (por ejemplo, requisitos puntuales del SFA que sean estrictamente mayores que el FAPI 2.0 base).

2. Soporte mTLS 1.3

2.1 Contexto técnico

Mutual TLS es la autenticación **bidireccional** a nivel TLS: además de validar el certificado del servidor, el cliente presenta un certificado X.509 firmado por una CA confiable.

En el ecosistema FAPI/Open Finance se usa para:

1. **Autenticación del cliente OAuth** (`tls_client_auth` o `self_signed_tls_client_auth` , RFC 8705 §2).
2. **Sender-Constrained Access Tokens**: el AS calcula `cnf.x5t#S256` (hash SHA-256 del cert) y el RS exige que el token solo sea usable sobre una conexión TLS donde el cliente presenta ese mismo cert (RFC 8705 §3).

TLS 1.3 (RFC 8446) elimina algoritmos débiles (RC4, SHA-1, MD5, CBC, RSA-KEX, renegociación) y mTLS sobre TLS 1.3 se solicita mediante `post_handshake_auth` .

Diagrama



2.2 ¿Obligatorio?

Sí. FAPI 2.0 exige *sender-constrained access tokens* y la única opción además de DPOP es mTLS. TLS 1.3 no es estrictamente exigido por la RFC pero la CMF requiere **TLS 1.2 mínimo, TLS 1.3 recomendado**; varias

guías chilenas marcan TLS 1.3 como obligatorio para 2026+.

2.3 Soporte Keycloak

✓ Soportado. Keycloak corre sobre **Quarkus + Netty** y soporta TLS 1.3 (Java 17/21). El *handshake* mTLS puede:

- Terminarse en el propio Keycloak (`https-client-auth=request|required`).
- Terminarse en un **reverse-proxy** (NGINX/HAProxy/Envoy) que reenvía el certificado por `X-SSL-Client-Cert` → habilitar `spi-x509cert-lookup-provider=nginx|haproxy|apache` .

```
# Terminación directa
https-client-auth=request
https-trust-store-file=/etc/keycloak/truststore.p12
https-trust-store-password=...

# Terminación en NGINX
spi-x509cert-lookup-provider=nginx
```

En el cliente OAuth (Realm → Clients → Credentials):

- Client Authenticator: `X.509 Certificate (tls_client_auth)` o `Self-signed Certificate (self_signed_tls_client_auth)` .

2.4 Recomendaciones si no soportado

No aplica (sí está soportado). Buenas prácticas:

- Forzar TLS 1.3 only en el `listen` del *edge*.
- Suite cifrada: `TLS_AES_128_GCM_SHA256` , `TLS_AES_256_GCM_SHA384` , `TLS_CHACHA20_POLY1305_SHA256` .
- Habilitar `ssl_verify_client on` y `ssl_verify_depth ≥ 2` (sub-CA del SFA).

3. Algoritmos de firma ES256 / PS256

3.1 Contexto técnico

FAPI 2.0 restringe los algoritmos JWS válidos a:

- **PS256** – RSASSA-PSS con SHA-256 y MGF1.
- **ES256** – ECDSA con curva P-256 y SHA-256.
- (Opcionalmente **EdDSA** Ed25519 en el último draft.)

Se prohíben: `none` , `HS*` (HMAC – simétrico), `RS256` (RSA-PKCS#1 v1.5 – maleable).

Los componentes que **deben** firmarse con estos algoritmos son: `request_object` (JAR), `client_assertion` (`private_key_jwt`), `id_token` , `JARM response` , `software_statement` (SSA), `DPoP proof` .

3.2 ¿Obligatorio?

Sí. FAPI 2.0 Security Profile §5.3.1 (Authorization Server) enumera los valores admisibles para `id_token_signing_alg_values_supported`.

3.3 Soporte Keycloak

✓ Soportado. Se configura por cliente:

```
Clients → <cliente> → Advanced → Fine Grain OpenID Connect Configuration
- ID Token Signature Algorithm: PS256 / ES256
- User Info Signed Response Alg: PS256
- Request Object Signature Algorithm: PS256
- Token Endpoint Auth Signing Alg: PS256
```

A nivel de Realm: Realm Settings → Keys → generar key providers `rsa-enc-generated` (PS256) y `ecdsa-generated` (P-256, alg=ES256).

El profile `fapi-2-security-profile` ya **fuerza** estas restricciones por *Executor* (`secure-signature-algorithm-executor`).

3.4 Recomendaciones

- Usar Hardware Security Modules (HSM) vía PKCS#11 (`KC_DB_KEYSTORE_TYPE=pkcs11`) para custodia de la clave privada que firma `id_token` y JARM.
- Rotar las keys cada 12 meses como máximo (Realm Settings → Keys → Active key → priority, marcar la anterior como `passive`).

4. Rotación de Refresh Tokens

4.1 Contexto técnico

Refresh Token Rotation (RTR): cada vez que un cliente intercambia un `refresh_token` por un nuevo `access_token`, el AS emite también un **nuevo** `refresh_token` e **invalida** el anterior. Si el anterior se reutiliza, el AS asume **token theft** y revoca toda la familia (reuse detection).

```
RT1 —/token grant=refresh_token→ AS → AT2 + RT2 (RT1 invalido)
RT1 (reusado por atacante) → AS → 400 invalid_grant + REVOCAR familia
```

Es alternativa al modelo *sender-constrained refresh token* (mTLS/DPoP-bound).

4.2 ¿Obligatorio?

Sí, **condicional**. FAPI 2.0 §5.3.2.2 exige que los `refresh_token`:

MUST be sender-constrained using either mTLS or DPOP, OR MUST be rotated on each use.

En SFA Chile lo correcto es **ambos**: sender-constrained **y** rotación (defensa en profundidad).

4.3 Soporte Keycloak

✓ Soportado de forma nativa.

```
Realm Settings → Tokens
☑ Revoke Refresh Token      (activa la rotación + reuse detection)
Refresh Token Max Reuse: 0
SSO Session Idle: 30 min
Client Session Max: 1 hour
```

4.4 Recomendaciones

- Mantener `Refresh Token Max Reuse = 0`.
- Definir `Refresh Token Lifetime` $\leq$ duración de consentimiento (en SFA: 180 días con re-confirmación a 90).
- Auditar evento `REFRESH_TOKEN_ERROR` para detectar reusos sospechosos (alimentar SIEM).

5. Inhabilitar flujos OAuth2 legacy

5.1 Contexto técnico

Flujos a **prohibir**:

Flow	Por qué
Resource Owner Password Credentials (ROPC)	Cliente maneja la contraseña $\Rightarrow$ phishing, sin MFA, sin federación.
Implicit (<code>response_type=token</code>)	Token expuesto en URL fragment, sin PKCE, sin sender-constraint, no replay-proof.
Hybrid (<code>code id_token</code> , <code>code token</code>)	No usado por FAPI 2.0 (sí FAPI 1.0); aún válido sólo si Message Signing y JAR estricto, FAPI 2.0 lo elimina.
Authorization Code sin PKCE	Vulnerable a robo de <code>code</code> en clientes públicos.
Device Code (para clientes web)	No es el caso de uso financiero.

FAPI 2.0 únicamente permite **Authorization Code + PKCE + PAR**, y opcionalmente **CIBA** para flujos *decoupled*.

5.2 ¿Obligatorio?

Sí.

5.3 Soporte Keycloak

✓ Soportado.

Vía *Client Policies* (`fapi-2-security-profile` ya lo aplica) y manualmente:

```
Clients → <cliente> → Settings
Standard Flow Enabled      : ON   (auth code)
Direct Access Grants Enabled : OFF (deshabilita ROPC)
Implicit Flow Enabled      : OFF
Service Accounts Enabled   : ON   (solo para client_credentials machine-to-machine)
```

Y un **Executor** del profile:

```
secure-response-type-executor:
  allowed-response-types: ["code"]
secure-grant-types-executor:
  allowed-grant-types: ["authorization_code", "refresh_token", "client_credentials"]
```

5.4 Recomendaciones

- Aplicar las *Client Policies* globalmente por defecto (matcher `any-client`).
- Auditar al menos mensualmente el listado de *clients* para detectar habilitación accidental de Implicit/ROPC.

6. Sender-Constrained Tokens

6.1 Contexto técnico

Un *access_token* tradicional ("bearer") es portador: cualquiera que lo posea lo usa. Un token **sender-constrained** sólo es válido si el llamante demuestra poseer una **clave privada** asociada al token (Proof-of-Possession).

Dos mecanismos estandarizados:

1. **mTLS-bound** (RFC 8705): el AS embebe `cnf: { "x5t#S256": "<hash-cert>" }` en el token. El RS exige que la conexión TLS use el mismo cert.
2. **DPoP-bound** (RFC 9449): el AS embebe `cnf: { "jkt": "<jwk-thumbprint>" }` . El cliente envía en cada request un header `DPoP: <jws>` firmado con la misma clave.

Diagrama (mTLS-bound)

```
AT = { iss, sub, exp, scope, cnf: { "x5t#S256": "AB12..." } }
```

↑
SHA-256(client TLS cert)

RS valida:

- firma AT
- `SHA-256(cliente_TLS_cert_actual) = cnf.x5t#S256`

Condicional. FAPI 2.0 acepta mTLS o DPoP. Se considera obligatorio "uno de los dos". SFA Chile menciona que para clientes con app móvil pública, **DPoP es la vía obligatoria**.

7.3 Soporte Keycloak

✓ Desde Keycloak 24 como *preview*, GA en 26.x.

Habilitar:

```
kc.sh start --features=dpop
```

Por cliente:

```
Clients → <client> → Advanced → DPoP Bound Access Tokens: ON  
→ DPoP nonces: enabled (opcional, anti-replay reforzado)
```

7.4 Recomendaciones

- Activar **DPoP Nonces** (`Use-DPoP-Nonce`) ⇒ obliga al cliente a incluir `nonce` provisto por el AS/RS, mitiga pre-computación.
- Configurar tiempo de tolerancia (`dpop-iat-leeway`) $\leq 60s$.
- En clientes móviles, persistir la clave DPoP en *secure enclave* (iOS Keychain / Android Keystore con `setUserAuthenticationRequired(true)`).

8. DCR (Dynamic Client Registration)

8.1 Contexto técnico

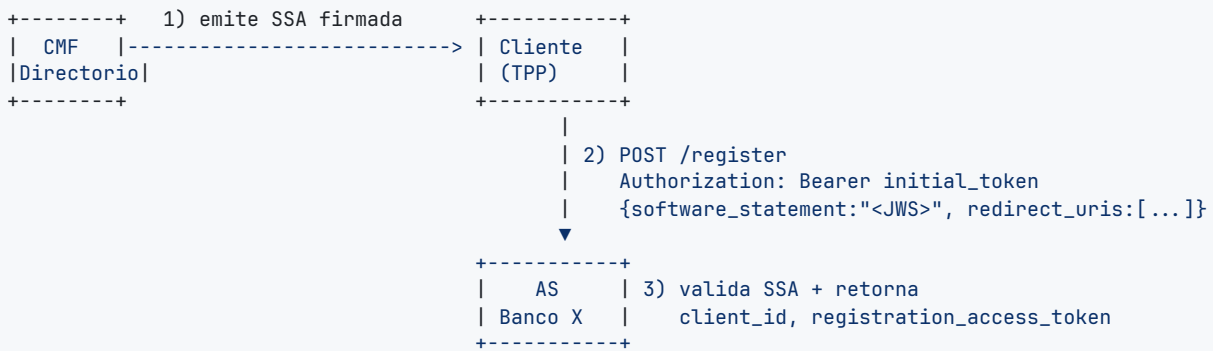
DCR (RFC 7591/7592) permite que un cliente OAuth se registre **programáticamente** en el AS, recibiendo `client_id` / `client_secret` (o `client_certificate`) sin intervención manual.

En Open Finance, DCR es la única vía: el "Directorio" del SFA emite **SSA** firmadas que el cliente entrega al AS de cada Institución Financiera; el AS valida el SSA y crea el registro.

Endpoints

- `POST /register` (RFC 7591): registro inicial.
- `GET/PUT/DELETE /register/{client_id}` (RFC 7592): mantenimiento, autenticado con `registration_access_token`.

Diagrama



8.2 ¿Obligatorio?

Sí para participar del SFA Chile.

8.3 Soporte Keycloak

☒ Parcial.

Keycloak soporta DCR RFC 7591/7592 con:

Realm Settings → Client Registration → Initial **access** tokens
→ **Trusted** hosts
→ Policies (validators)

Pero **no valida nativamente el `software_statement` SSA** según las claves públicas del Directorio del SFA y sus claims específicos. Para eso se requiere implementar un **`ClientRegistrationPolicy` SPI**.

8.4 Recomendaciones (cuando soporte parcial)

Implementar un SPI personalizado:

```
public class SfaSoftwareStatementPolicy implements ClientRegistrationPolicy {
    public void beforeRegister(ClientRegistrationContext ctx) {
        String ssaJws = ctx.getClient().getSoftwareStatement();
        JWSInput input = new JWSInput(ssaJws);
        JWKSet directoryJwks = fetchJwks("https://directorio.cmf.cl/jwks");
        if (!RSAProvider.verify(input, directoryJwks)) throw new ClientRegistrationException("SSA inválido");
        JsonNode claims = JsonSerialization.readValue(input.getContent(), JsonNode.class);
        if (!claims.get("software_environment").asText().equals("production")) throw ...;
        // Copiar org_id, software_id, redirect_uris desde el SSA a los attrs del cliente
    }
}
```

Empaquetar como JAR en `/opt/keycloak/providers/` y registrar:

META-INF/services/org.keycloak.services.clientregistration.policy.ClientRegistrationPolicyFactory

Y activar la policy en Realm → Client Registration → Policies.

9. Consumo de Software Statements (SSA)

9.1 Contexto técnico

Un **Software Statement Assertion** es un JWT firmado por una autoridad central (en Chile: el **Directorio del SFA / CMF**) que **declara** la identidad y atributos de un *software* cliente:

```
{
  "iss": "https://directorio.cmf.cl",
  "iat": 1717000000,
  "exp": 1748536000,
  "jti": "ssa-9d8e...",
  "org_id": "12345-6",
  "org_name": "Banco Ejemplo S.A.",
  "software_id": "tpp-app-v1",
  "software_client_name": "MiBancoApp",
  "software_jwks_uri": "https://tpp.example.cl/jwks",
  "software_redirect_uris": ["https://tpp.example.cl/cb"],
  "software_roles": ["AISP", "PISP"],
  "software_environment": "production"
}
```

El AS lo recibe en el endpoint DCR (`software_statement`) y lo valida:

1. Firma JWS con clave del Directorio (publicada en `/jwks`).
2. `iss` corresponde al Directorio confiable.
3. `exp` no vencido.
4. `org_id` / `org_status` no esté en lista de revocación.

9.2 ¿Obligatorio?

Sí para SFA Chile.

9.3 Soporte Keycloak

✗ No nativo.

Keycloak conoce el parámetro `software_statement` y lo guarda como atributo, pero **no valida su firma ni claims** contra el Directorio. Se requiere extensión.

9.4 Recomendaciones

- Implementar el SPI de la sección 8.4 con validación completa.
 - Cachear el JWKS del Directorio con $TTL \leq 1$ h.
 - Implementar verificación contra la **lista de organizaciones revocadas** (endpoint del Directorio del SFA).
 - Persistir el `jti` del SSA para evitar replay (`software_statement` reusado).
-

10. JARM (JWT Secured Authorization Response Mode)

10.1 Contexto técnico

En OAuth tradicional, la respuesta de autorización (`code` , `state`) viaja como query parameter en el redirect:

```
https://client/cb?code=xyz&state=abc
```

Esto es manipulable y, peor, *non-repudiable* (no firmado). **JARM** lo reemplaza por un **JWT firmado** y entregado como un único parámetro `response` :

```
https://client/cb?response=eyJhbGciOiJIUzI1NiIsImtpZCI6Ii4uIn0...
```

El JWT contiene:

```
{
  "iss": "https://as.banco.cl",
  "aud": "client-123",
  "exp": 1717000600,
  "code": "xyz",
  "state": "abc"
}
```

Modos: `response_mode=query.jwt` | `fragment.jwt` | `form_post.jwt` | `jwt` .

10.2 ¿Obligatorio?

Obligatorio en FAPI 2.0 Message Signing. Recomendado en Security Profile. Para flujos de pago en SFA Chile, **obligatorio**.

10.3 Soporte Keycloak

✅ Soportado desde Keycloak 21 (preview), GA en 24.x.

Habilitar:

```
kc.sh start --features=jarm
```

Y por cliente:

```
Clients → <client> → Advanced → Authorization Signed Response Alg: PS256
                                → Authorization Response Mode: jwt
```

10.4 Recomendaciones

- Combinar JARM con **PAR** para firmar tanto el request como la response (cierre end-to-end de la cadena).
- Usar `form_post.jwt` cuando el JWT pueda exceder el límite de URL (~2KB).

11. PAR Obligatorio (Pushed Authorization Requests)

11.1 Contexto técnico

PAR (RFC 9126) elimina el envío de parámetros sensibles vía *front-channel* (URL del navegador). El cliente hace un `POST` al endpoint `/par` autenticado, el AS guarda los parámetros y devuelve un `request_uri` opaco:

1) <code>POST /par (mTLS)</code> <code>client_id=...</code> <code>redirect_uri=...</code> <code>scope=...</code> <code>code_challenge=...</code> <code>response_type=code</code> <code>state=...</code> → 201 { "request_uri": "urn:..abc", "expires_in": 60 }	2) <code>Redirect /authorize?client_id=..&request_uri=urn:ietf:params:oauth:request_uri=..</code> ↓ AS recupera la request guardada por <code>request_uri</code> ↓ [Auth + Consent + redirect a callback]
---	---

Beneficios:

- Imposible alterar parámetros vía URL (CSRF de autorización).
- Front-channel sólo lleva `client_id` + `request_uri` opacos.
- Permite *request objects* (JAR) grandes sin chocar con límites de URL.

11.2 ¿Obligatorio?

Sí. FAPI 2.0 §5.3.1.1: AS *MUST* require PAR for all authorization requests.

11.3 Soporte Keycloak

✅ Soportado nativamente.

Habilitar feature `par`. Por defecto el endpoint queda en `/realms/{realm}/protocol/openid-connect/ext/par/request`.

Realm Settings → OpenID Endpoint Configuration → `"pushed_authorization_request_endpoint"`

Por cliente / vía profile:

`secure-par-executor`: (lo activa `fapi-2-security-profile`)

11.4 Recomendaciones

- En el cliente, **siempre** firmar el contenido como `request_object` (JAR) además del POST, para garantizar integridad incluso si el `request_uri` se filtrara.
- TTL de `request_uri` $\leq 90s$ (recomendación FAPI WG).

12. PKCE Obligatorio (S256)

12.1 Contexto técnico

PKCE (RFC 7636) protege el *authorization code* contra interceptación:

1. Cliente genera `code_verifier` (43-128 chars random).
2. `code_challenge` = `BASE64URL(SHA-256(code_verifier))`.
3. Envía `code_challenge` + `code_challenge_method=S256` en la request de autorización.
4. Al canjear el code, envía `code_verifier`.
5. AS comprueba `SHA-256(verifier) = challenge`.

```
code_verifier      = "dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWF0EjXk"
code_challenge_S256= "E9MeIhoa20wvFrEMTJguCHaoeK1t8URWbuGJSstw-cM"
```

FAPI 2.0 obliga **únicamente** S256 (no plain).

12.2 ¿Obligatorio?

Sí.

12.3 Soporte Keycloak

✓ Soportado nativamente.

Por cliente:

```
Clients → Advanced → Proof Key for Code Exchange Code Challenge Method: S256
```

El profile FAPI 2 Security activa el executor `pkce-enforcer-executor`.

12.4 Recomendaciones

- Validar que `code_verifier` tenga **mínimo 43 caracteres** (la RFC lo exige).
- Verificar que aplicaciones móviles **no** registren el `code_verifier` en logs ni en *URL-schemes intent* (Android intent-filter sin verificación).

13. OCSP (Online Certificate Status Protocol)

13.1 Contexto técnico

OCSP (RFC 6960) permite consultar en tiempo real el estado de un certificado X.509 (Good / Revoked / Unknown) contra el responder de la CA:



La respuesta es un `BasicOCSPResponse` firmado por la CA, contiene `thisUpdate`, `nextUpdate`, `certStatus`.

En FAPI/Open Finance se valida el cert presentado por el cliente vía mTLS contra OCSP **en cada conexión** (o cacheado por `nextUpdate`).

13.2 ¿Obligatorio?

Sí. La cadena PKI del SFA exige verificar revocación; OCSP (preferentemente) o CRL como fallback.

13.3 Soporte Keycloak

⚠ Parcial.

Keycloak hace validación de revocación mediante el **X.509 Client Certificate Authenticator** (`x509-direct-grant-authenticator` / `x509-browser-authenticator`):

```
Authentication → Flows → X.509 Direct Grant → Config
CRL Checking Enabled : ON
OCSP Checking Enabled : ON
OCSP Responder URI    : https://ocsp.directorio.cmf.cl
OCSP Responder Cert   : (X.509 PEM)
```

Limitación: la validación OCSP de Keycloak es sobre el cert presentado a su *X509 Authenticator* (login con cert), **no** automáticamente sobre cada token-endpoint mTLS — eso depende del *cert-lookup provider* y la JVM. Se recomienda configurar a nivel de JVM:

```
-Dcom.sun.security.enableCRLDP=true
-Dcom.sun.net.ssl.checkRevocation=true
-Docsp.enable=true
-Docsp.responderURL=https://ocsp.directorio.cmf.cl
```

13.4 Recomendaciones

- Implementar un `X509ClientCertificateAuthenticator` **SPI custom** que, además de la validación PKIX, llame al responder OCSP del SFA, valide su firma con el cert del responder, y aplique caché con

`nextUpdate` .

- Métricas Prometheus de tasa de fallas OCSF para alertar cuando responder esté caído (decidir *fail-open* vs *fail-closed* según política).

14. OCSF Stapling

14.1 Contexto técnico

En vez de que el AS (verificador) consulte OCSF, es el **servidor TLS** quien adjunta (`staples`) la respuesta OCSF firmada en el handshake (TLS Certificate Status Request, RFC 6066). Beneficios:

- Reduce latencia (no hay round-trip extra).
- Mejora privacidad (la CA no ve qué clientes consultan al servidor).
- Funciona aunque el OCSF esté momentáneamente offline (TTL).

Variante: **OCSF Must-Staple** (RFC 7633) — el cert tiene una extensión que **obliga** al server a staplear; si falta, el cliente rechaza.

14.2 ¿Obligatorio?

Recomendado. SFA Chile lo exige *para el certificado del propio AS* (server-side stapling); para el lado *cliente* (cert del TPP), OCSF "tradicional" sigue siendo lo normal.

14.3 Soporte Keycloak

⚠ Vía reverse-proxy.

El stack Quarkus de Keycloak no expone configuración directa de OCSF stapling. La práctica habitual es:

- Terminar TLS en **NGINX**:

```
ssl_stapling on;
ssl_stapling_verify on;
resolver 8.8.8.8 valid=60s;
ssl_trusted_certificate /etc/nginx/ca-chain.pem;
```

- O en **HAProxy** (`set ssl ocsf-response`).
- O en **Envoy** (`ocsf_staple_policy: must_staple`).

14.4 Recomendaciones

- Refrescar la respuesta OCSF cada 4 h (los stapled responses suelen tener TTL de 7 días).
 - Monitorear que `ssl_stapling` esté efectivamente activo (no todos los responders lo soportan; usar `openssl s_client -status -connect ...`).
-

15. Consentimiento Granular e Informativo

15.1 Contexto técnico

El consentimiento debe ser:

- **Granular:** cada *scope* / cada *recurso legal* aprobado individualmente.
- **Informativo:** explicación clara al usuario de qué dato se compartirá, con quién, por cuánto tiempo, para qué fin.
- **Auditable:** trazabilidad con identificador único (`consent_id`), fecha, hash de los términos mostrados.
- **Revocable:** en cualquier momento por el usuario, propagando revocación a tokens activos.

Ejemplo SFA: un usuario otorga acceso a:

- Cuentas (`accounts:read`) por 12 meses.
- Saldo (`balances:read`) por 12 meses.
- Movimientos últimos 90 días (`transactions:read?from=2025-08-25&to=2025-11-25`).
- **Pero NO** acepta `direct-debits:write` .

15.2 ¿Obligatorio?

Sí. SFA Chile exige consentimiento explícito por *recurso* y por *propósito*.

15.3 Soporte Keycloak

⚠ Parcial.

Keycloak tiene una *consent screen* configurable (`Client Scopes` → `consent screen text`) pero:

- No soporta nativamente parámetros dinámicos (rango de fechas, cuentas específicas).
- No persiste un `consent_id` enriquecido (solo lista de scopes aceptados por cliente).
- Revocación: existe `RevocationEndpoint` para tokens, y "consent revoke" en Account Console, pero sin propagar a un servicio externo.

15.4 Recomendaciones

Patrón recomendado en Open Finance (similar a Brasil):

1. **Pre-creación de consentimiento** fuera de Keycloak: el TPP llama a un *Consent API* del banco que crea un `consent_id` con detalles completos.
2. El TPP pasa el `consent_id` en `authorization_details` (RAR – ver §16) o en un scope dinámico (`consent:<consent_id>`).
3. Keycloak renderiza la pantalla de consentimiento delegando en una **Authenticator SPI** que consulta el Consent API y muestra los datos enriquecidos al usuario.
4. Al aceptar, se asocia el `consent_id` al token (claim `consent_id`).
5. Revocación: actualizar Consent API + invocar `RevocationEndpoint` de Keycloak para los tokens vivos.

16. Rich Authorization Requests (RAR – RFC 9396)

16.1 Contexto técnico

RAR introduce un nuevo parámetro `authorization_details` (JSON) que reemplaza al limitado `scope` :

```
"authorization_details": [
  {
    "type": "payment_initiation",
    "locations": ["https://api.banco.cl/payments"],
    "instructedAmount": {"currency": "CLP", "amount": "150000"},
    "creditorAccount": {"iban": "CL0000123..."},
    "creditorName": "Juan Pérez"
  },
  {
    "type": "account_information",
    "locations": ["https://api.banco.cl/accounts"],
    "accounts": ["CL0099887..."],
    "permissions": ["ReadBalances", "ReadTransactionsBasic"]
  }
]
```

Permite expresar *exactamente* qué se autoriza (no sólo "leer cuentas" sino "leer esta cuenta específica, sólo balances, sólo este día").

16.2 ¿Obligatorio?

Opcional en FAPI 2.0 base, obligatorio en SFA Chile para *payment initiation* y otros casos de uso de escritura.

16.3 Soporte Keycloak

⚠ Parcial.

Keycloak puede transportar `authorization_details` como claim en el `access_token` (con un *Protocol Mapper* tipo *Hardcoded/Script*), pero **no lo procesa semánticamente** (no valida `type` , no lo muestra en `consent screen` estructurado).

16.4 Recomendaciones

- Implementar un `AuthorizationDetailsValidator` **SPI** que:
 - Reconozca los `type` válidos definidos por el SFA Chile (`payment_initiation` , `account_information` , etc.).
 - Valide el esquema JSON de cada tipo.
 - Inyecte el contenido en `id_token` y `access_token` como claim `authorization_details` .

- Pantalla de consent que renderice los `authorization_details` en lenguaje humano ("Vas a autorizar el pago de \$150.000 a Juan Pérez desde tu cuenta XYZ").

17. Mapeo de Scopes y Recursos Legales

17.1 Contexto técnico

Cada *scope* OAuth o *type* RAR debe estar **mapeado** a:

- Recurso legal (Ej: "Ley 21.521 art. 16 letra c – datos de cuentas").
- Finalidad declarada al usuario.
- TTL máximo permitido por norma.
- Roles del cliente (AISP / PISP) habilitados.

Tabla típica:

Scope	Recurso Legal	Roles	TTL máx	Sensibilidad
<code>accounts:read</code>	Art. 16(c) NCG 514	AISP	365d	Media
<code>balances:read</code>	Art. 16(c) NCG 514	AISP	365d	Media
<code>transactions:read</code>	Art. 16(d) NCG 514	AISP	90d	Alta
<code>payments:write</code>	Art. 17 NCG 514	PISP	1d (single)	Crítica

17.2 ¿Obligatorio?

Sí, como parte del cumplimiento normativo y de la documentación de evaluación de impacto en datos personales (Ley 19.628 reformulada).

17.3 Soporte Keycloak

✗ No soportado nativamente.

Keycloak conoce *Client Scopes* y permite metadata por scope, pero no expone modelos "recurso legal", "finalidad", "TTL por scope".

17.4 Recomendaciones

- Implementar un **catálogo central** (DB o configmap) de scopes ↔ recurso legal, mantenido por el equipo de Compliance.
- En el `Authenticator` que muestra el consent screen, leer el catálogo y desplegar el detalle (cumple además §15).
- En el `AccessTokenMapper`, embeber `legal_basis` y `purpose` como claims (útil para auditoría aguas abajo).
- En CI/CD, validar que todo scope nuevo en Keycloak tenga su entrada en el catálogo.

18.1 Contexto técnico

- No puedan ser modificados a posteriori sin detección (integridad).
- La autoría sea verificable (autenticidad).
- Idealmente, encadenados (hash chain estilo blockchain ligero) para detectar eliminaciones.

a) **Logs de verificación de certificados (mTLS / OCSP):** por cada handshake mTLS, registrar `client_cert_sha256`, `cn`, `serial`, `ocsp_status`, `ocsp_response_signature_valid`, `timestamp`.

Estructura recomendada (formato ETSI TS 119 512 o JOSE-firmado):

18.2 ¿Obligatorio?

18.3 Soporte Keycloak

- No los firma criptográficamente.
- No encadena hashes.
- No persiste eventos de bajo nivel como handshake mTLS (esos viven en el reverse-proxy o JVM).

18.4 Recomendaciones

Patrón propuesto:

1. Captura:

- Eventos Keycloak → `EventListenerProvider` SPI custom.
- Eventos mTLS → módulo de log de NGINX/Envoy enviando a Kafka.
- Eventos OCSP/SSA → desde los SPI propios (§13, §9).

2. Firma + cadena: servicio `log-signer` (microservicio) que:

- Recibe eventos vía Kafka topic `audit.raw`.
- Calcula `prev_hash` (consulta último evento por partition).
- Firma con clave en HSM (PS256).
- Publica en topic `audit.signed`.

3. Almacenamiento WORM: `audit.signed` → S3 con *Object Lock* compliance mode (o equivalente Object Storage on-prem) + replica a *time-stamping authority* TSA (RFC 3161) para sello de tiempo confiable.

4. Verificación: comando CLI / batch nocturno que recorre cadena y verifica firmas y hashes; alerta SIEM si rompe.

Stack sugerido: Keycloak SPI → Kafka → Flink/Quarkus signer → S3 (Object Lock) → ELK para indexación + Grafana para visualización.

19. Identificación del Issuer en respuestas (RFC 9207)

19.1 Contexto técnico

RFC 9207 — OAuth 2.0 Authorization Server Issuer Identification establece que el AS DEBE incluir el parámetro `iss` en la **respuesta de autorización** (la URL de redirección al `redirect_uri` del cliente). Su valor es el *issuer identifier* del AS (idéntico al del campo `iss` de los tokens y al `issuer` publicado en `/.well-known/openid-configuration`).

El objetivo es mitigar los **ataques de mix-up** (también conocidos como *IdP mix-up*) en clientes que aceptan múltiples Authorization Servers: sin `iss`, el cliente no puede determinar de **qué** AS proviene el `code` y un atacante puede inducir el canje del `code` en el AS equivocado.

Diferencia con JARM (sección 10)

- **RFC 9207**: agrega un parámetro **plano** `iss=...` junto a `code / state`. No firmado, simple.
- **JARM** (sección 10): empaqueta toda la respuesta como un **JWT firmado** cuyo claim obligatorio `iss` ya cubre RFC 9207.

JARM es un *superconjunto*: si el cliente recibe JARM, automáticamente satisface RFC 9207. En FAPI 2.0 Security Profile (sin JARM) el `iss` plano es **obligatorio**; en Message Signing (con JARM) está implícito.

Ejemplos

Respuesta sin JARM (cumple RFC 9207):

```
HTTP/1.1 302 Found
Location: https://tpp.example.cl/cb
        ?code=SpLxl0BeZQQYbYS6WxSbIA
        &state=af0ifjsldkj
        &iss=https%3A%2F%2Fas.banco.cl%2Frealms%2Fsfa
```

Respuesta con JARM (cubre RFC 9207 en el claim):

```
{
  "iss": "https://as.banco.cl/realms/sfa",
  "aud": "client-123",
  "exp": 1717000600,
  "code": "SpLxl0BeZQQYbYS6WxSbIA",
  "state": "af0ifjsldkj"
}
```

Diagrama (mix-up mitigado por `iss`)

```
Atacante AS-B — induce flujo —> Usuario — inicia en AS-A —> /authorize
                                     |
                                     | redirect con code
                                     v
                               Cliente revisa "iss":
                               - presente y = AS-A → OK, canjea code
                               - ausente o ≠ AS-A → ABORTA (mix-up detectado)
```

19.2 ¿Obligatorio?

Sí, en FAPI 2.0 Security Profile §5.3.2.2.7: el AS DEBE enviar `iss` en respuestas no-JARM. En Message Signing, la presencia del claim `iss` en el JARM JWT cumple el requisito.

19.3 Soporte Keycloak ≥ 26.x

✅ Soportado nativamente desde Keycloak 21. El parámetro `iss` se emite por defecto en respuestas de autorización si el realm tiene definido un Issuer URL válido (Realm Settings → Tokens). El *Client Profile* `fapi-2-security-profile` activa explícitamente el `executor consent-required-executor` y `secure-redirect-uris-enforcer`, y desde 24.x incluye `iss` por defecto.

```
Realm Settings → Tokens → Issuer URL: https://as.banco.cl/realms/sfa
```

Verificación rápida con `curl` :

```
curl -v "https://as.banco.cl/realms/sfa/protocol/openid-connect/auth?..."
# en el Location ha de aparecer &iss=https%3A%2F%2Fas.banco.cl%2Frealms%2Fsfa
```

19.4 Recomendaciones

- Verificar que `Issuer URL` coincide **exactamente** con el dominio público del AS (incluido `https://` , sin barra final).
- En despliegues multi-realm, validar que cada realm publica un `iss` distinto.
- Documentar el valor canónico de `iss` para que los TPPs lo whitelisten en su verificación.
- Si el AS está detrás de un reverse-proxy, fijar `KC_HOSTNAME` y `KC_HOSTNAME_STRICT=true` para evitar discrepancias.

20. JAR / Request Object firmado (RFC 9101)

20.1 Contexto técnico

RFC 9101 — JWT-Secured Authorization Request (JAR) define el parámetro `request` (o `request_uri`) cuyo valor es un **JWS** firmado por el cliente que **contiene** todos los parámetros del request de autorización (`response_type` , `client_id` , `redirect_uri` , `scope` , `state` , `code_challenge` , `nonce` , etc.).

Beneficios:

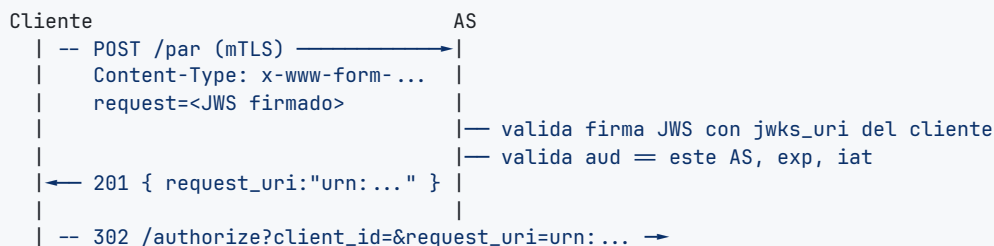
1. **Integridad:** los parámetros no pueden ser alterados por el navegador, proxies o el usuario.
2. **Autenticidad:** el AS confirma la identidad del cliente antes incluso de iniciar la autenticación del usuario.
3. **No-repudio:** el cliente firmó **exactamente** lo que pidió.
4. **Compatible con PAR:** el `request_object` se envía dentro del POST a `/par` , combinando confidencialidad (POST) e integridad (firma).

Estructura

```
Header (JWS):
{ "alg": "PS256", "kid": "client-key-1", "typ": "oauth-authz-req+jwt" }

Payload:
{
  "iss":          "client-123",           // = client_id
  "aud":          "https://as.banco.cl/realm/sfa",
  "exp":          1717000060,             // ≤ 60s desde iat
  "iat":          1717000000,
  "jti":          "01H...",
  "response_type": "code",
  "client_id":    "client-123",
  "redirect_uri": "https://tpp.example.cl/cb",
  "scope":        "openid accounts",
  "state":        "af0ifjsldkj",
  "code_challenge": "E9Me1hoa20wvFrEMTJguCHaoeK1t8URWbuGJSstw-cM",
  "code_challenge_method": "S256",
  "nonce":        "n-0S6_WzA2Mj"
}
```

Diagrama (JAR dentro de PAR)



20.2 ¿Obligatorio?

- **FAPI 2.0 Security Profile:** el cliente DEBE usar PAR; el cuerpo del POST puede contener parámetros sueltos o un `request_object`. La firma es **recomendada**.
- **FAPI 2.0 Message Signing:** el `request_object` (JAR) firmado es **OBLIGATORIO**. SFA Chile lo exige para todos los flujos de escritura (*payment initiation*).

20.3 Soporte Keycloak ≥ 26.x

✓ Soportado nativamente.

Configuración por cliente:

```

Clients → <cliente> → Advanced
Request Object Signature Algorithm : PS256
Request Object Required             : Request Object (with PAR)
Request URI Verification             : true
Valid request URIs                  : (vacío - solo se aceptan las generadas por PAR)
  
```

El profile `fapi-2-message-signing-profile` activa el executor `secure-request-object-executor` que verifica:

- Firma con alg permitido (PS256/ES256).
- `aud` == issuer del AS.
- `exp - iat` ≤ 60s.
- `nbf` opcional pero validado si presente.

20.4 Recomendaciones

- Combinar JAR **dentro** de PAR (no usar `request_uri` pre-registrado — ver sección 41).
- Validar que la JVM de Keycloak tenga BouncyCastle FIPS si se requiere certificación criptográfica.
- En el cliente, persistir `jti` para evitar reusos en ventana de `exp`.
- Rechazar todo request con `request_object` sin firma (`alg=none` está prohibido).

21. Client Authentication: `private_key_jwt` y `tls_client_auth`

21.1 Contexto técnico

FAPI 2.0 restringe los métodos de autenticación de cliente a **exactamente dos**:

1. `private_key_jwt` (RFC 7523): el cliente envía un JWT firmado con su clave privada en el parámetro `client_assertion`, con `client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer`.
2. `tls_client_auth` o `self_signed_tls_client_auth` (RFC 8705): el cliente se autentica mediante el certificado X.509 presentado en el handshake mTLS.

Quedan **prohibidos**:

- `client_secret_basic` (HTTP Basic) — secret transmitido por canal aunque sea TLS, susceptible a leaks.
- `client_secret_post` — idem.
- `client_secret_jwt` — HMAC con secret compartido, no permite *non-repudiation*.

Diagrama

```
private_key_jwt:
  POST /token
  grant_type=authorization_code&code=xyz
  &client_assertion_type=...jwt-bearer
  &client_assertion=eyJ...<JWS-firmado-con-clave-privada-del-cliente>...

tls_client_auth:
  (handshake mTLS con cert del cliente)
  POST /token
  grant_type=authorization_code&code=xyz&client_id=...
  (AS deriva la identidad del cliente desde la cadena de certs validada)
```

Estructura del `client_assertion`

```
Header: { "alg": "PS256", "kid": "client-key-1", "typ": "JWT" }
Payload:
{
  "iss": "client-123",           // = client_id
  "sub": "client-123",          // = client_id
  "aud": "https://as.banco.cl/realms/sfa/protocol/openid-connect/token",
  "exp": 1717000060,
  "iat": 1717000000,
  "jti": "01H..."             // único, persistido contra replay
}
```

21.2 ¿Obligatorio?

Sí. FAPI 2.0 Security Profile §5.3.2.2: `token_endpoint_auth_methods_supported` debe limitarse a estos dos. Aplica a todos los endpoints autenticados: `/token`, `/par`, `/register`, `/revoke`, `/introspect`.

21.3 Soporte Keycloak ≥ 26.x

✓ Soportado nativamente.

Por cliente:

```
Clients → Credentials → Client Authenticator:
- "Signed JWT"      (private_key_jwt)
- "X509 Certificate" (tls_client_auth)
- "X509 Self-signed" (self_signed_tls_client_auth)
```

El profile FAPI 2.0 incluye el executor `client-authn-executor` que rechaza cualquier cliente nuevo o existente que tenga configurado un método distinto.

21.4 Recomendaciones

- Custodiar la clave privada del `private_key_jwt` en HSM (sección 44).
- Definir TTL del `client_assertion` $\leq 60s$, validar `aud` *exacto* al endpoint (no sólo al issuer).
- Persistir `jti` recibidos en cache distribuida (Infinispan) para detectar replay.
- Para `tls_client_auth`, mantener el campo `tls_client_auth_subject_dn` o `tls_client_auth_san_dns` definido en el cliente y validar coincidencia con el cert presentado.

22. JWKS rotativo del cliente (`jwks_uri`)

22.1 Contexto técnico

Para que un cliente pueda rotar sus claves sin re-registrarse en cada AS, FAPI 2.0 exige que **publique sus claves públicas** en un endpoint **HTTPS** denominado `jwks_uri`, en formato **JWK Set** (RFC 7517):

```
GET https://tpp.example.cl/.well-known/jwks.json
→
{
  "keys": [
    { "kty": "RSA", "use": "sig", "alg": "PS256", "kid": "client-key-2026Q2",
      "n": "...", "e": "AQAB" },
    { "kty": "EC", "use": "enc", "alg": "ECDH-ES+A256KW", "kid": "client-enc-1",
      "crv": "P-256", "x": "...", "y": "..." }
  ]
}
```

El AS **debe cachear** el JWKS con TTL acotado (1h típico) y, ante un `kid` desconocido, hacer **re-fetch** inmediato para soportar rotación *just-in-time*.

Flujo de rotación

```
T+0      : cliente publica key K2 junto a K1 en jwks_uri (overlap)
T+0..T+1h : cliente sigue firmando con K1
T+1h     : cliente comienza a firmar con K2
T+1h+1d  : cliente retira K1 del jwks_uri
```

22.2 ¿Obligatorio?

Sí, FAPI 2.0 prohíbe el registro estático de `jwks` (claves embebidas en el cliente). Sólo se admite `jwks_uri` (RFC 7591 §2 dice "either ... or", FAPI lo restringe).

22.3 Soporte Keycloak ≥ 26.x

✓ Soportado.

```
Clients → <cliente> → Keys → "Use JWKS URL": ON
→ JWKS URL: https://tpp.example.cl/.well-known/jwks.json
```

Caché TTL: configurable en `spi-keys-jwks-cache-...` (default 1h). Re-fetch ante `kid` desconocido: comportamiento por defecto desde 24.x.

22.4 Recomendaciones

- Servir el `jwks_uri` sobre **mTLS opcional** o al menos sobre TLS con OCSP stapling.
- TTL de caché $\leq 1h$.
- Monitor de availability del `jwks_uri` (alarma si caído más de 5 min — bloquea logins).
- Política de overlap: nueva clave publicada **antes** de usarse, vieja retirada **después** de cesar uso (24h overlap mínimo).
- Validar que la respuesta sea `application/jwk-set+json` y JSON canónico.

23. `aud` / `exp` estrictos en `client_assertion`

23.1 Contexto técnico

El `client_assertion` (sección 21) es un JWT corto firmado por el cliente. Si las validaciones son laxas, abre vectores:

Validación débil	Vector
<code>aud</code> = issuer (no endpoint)	Replay del assertion en otro endpoint del mismo AS
<code>aud</code> no validado	Phishing: AS malicioso captura el assertion y lo reusa contra el AS real
<code>exp</code> largo (> 5 min)	Replay si se filtra
<code>jti</code> no persistente	Replay dentro de la ventana de <code>exp</code>
<code>iat</code> no validado	Aceptar assertions del futuro o muy antiguos

Validaciones obligatorias

- `iss` = `sub` = `client_id` (registrado y activo)
- `aud` = URL EXACTA del endpoint llamado (no sólo el issuer)
- `exp` - `iat` ≤ 60 segundos
- `exp` en el futuro (con leeway $\leq 5s$)
- `iat` no más antiguo que (now - 60s)
- `jti` único en cache distribuida con TTL $\geq exp$
- `alg` $\in \{ PS256, ES256 \}$ (rechazar HS*, RS256, none)

23.2 ¿Obligatorio?

Sí. FAPI 2.0 §5.3.2.2 + RFC 7523 §3.

23.3 Soporte Keycloak ≥ 26.x

✓ Soportado nativamente. Por defecto Keycloak valida `iss / sub / aud / exp / iat / jti`.

Detalles:

- `aud` se valida contra el endpoint llamado (no sólo issuer) desde 22.x.
- `jti` se persiste en Infinispan; en cluster requiere replicación habilitada del cache `clientSessions` o un cache dedicado. Verificar:

```
kc.sh start --cache=ispn --cache-config-file=cache-ispn.xml
# y en el XML: <distributed-cache name="actionTokens" owners="2"/>
```

- Tolerancia `iat / exp` configurable: `--spi-client-jwt-leeway=5`.

23.4 Recomendaciones

- Validar manualmente con la conformance suite el caso de `aud=issuer` (debe rechazar) vs `aud=token_endpoint` (debe aceptar).
- En cluster multi-site, replicar el cache `jti` cross-site (alta latencia tolerable porque `exp ≤ 60s`).
- Métrica Prometheus por tasa de rechazos por `jti` duplicado (detecta replay activo).
- Rechazar `nbft` futuros.

24. CIBA Backchannel (OIDC Client-Initiated Backchannel Authentication)

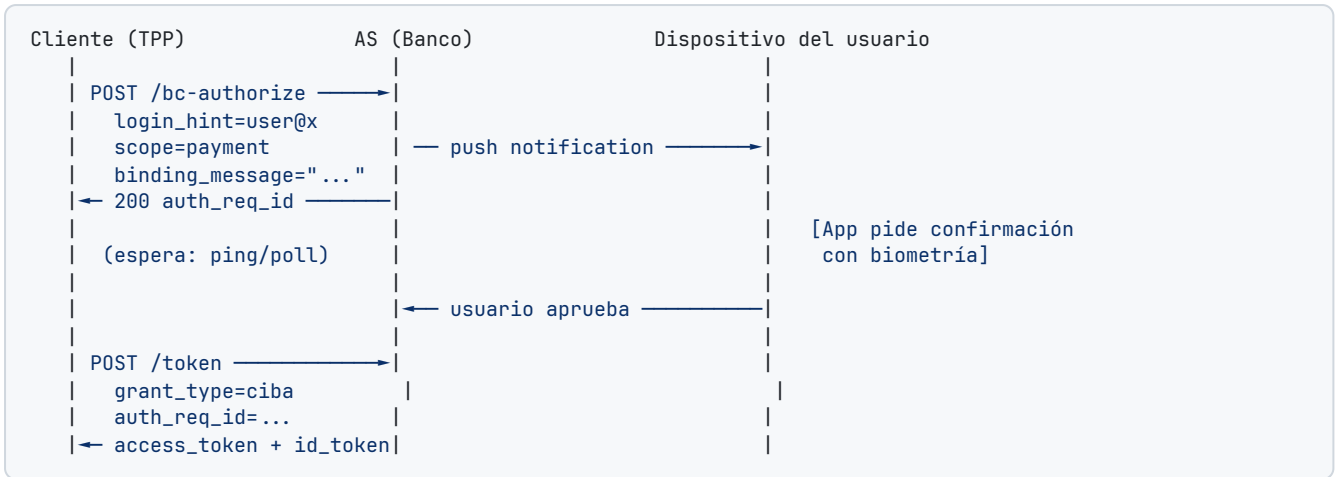
24.1 Contexto técnico

CIBA permite flujos **decoupled**: el cliente (TPP, comercio, agente IVR) **inicia** la autenticación, pero el usuario la confirma en un **dispositivo separado** (típicamente la app del banco que ya lo tiene autenticado con biometría).

Útil para:

- Pagos iniciados en POS / e-commerce, autorizados en el celular del usuario.
- Centros de llamadas: agente solicita consentimiento, usuario aprueba en su app.
- Asistentes de voz / IoT.

Diagrama



Parámetros clave

Param	Descripción
login_hint / login_hint_token / id_token_hint	Cómo identifica al usuario
binding_message	Texto humano corto mostrado en ambos extremos (anti-substitution)
user_code	OTP opcional ingresado por el usuario al confirmar
requested_expiry	TTL del request, default 300s

24.2 ¿Obligatorio?

- **FAPI 2.0**: opcional, pero hay un perfil específico **FAPI-CIBA** que lo formaliza.
- **SFA Chile**: obligatorio para casos *decoupled* específicos (iniciación de pagos desde canales no web).

24.3 Soporte Keycloak ≥ 26.x

✓ Soportado vía feature flag:

```
kc.sh start --features=ciba,fapi-2,par,dpop
```

Configuración por cliente:

```
Clients → <cliente> → Advanced
CIBA Grant Enabled           : ON
Backchannel Token Delivery Mode : ping | poll | push
Backchannel Client Notification Endpoint : https://tpp.example.cl/ciba-cb
```

Authentication Flow CIBA: Authentication → Flows → Direct Grant → Add CIBA .

24.4 Recomendaciones

- Combinar con WebAuthn/FIDO2 (sección 27) para la confirmación en el dispositivo (no SMS-OTP).

- Mostrar `binding_message` en **ambos** extremos (TPP y app del banco) — el usuario debe ver el mismo texto.
- Limitar `requested_expiry` $\leq 120s$ en pagos.
- Auditar latencia desde `bc-authorize` hasta confirmación; alertar si $> 60s$ (UX y posible spoofing).

25. CIBA Delivery Modes (`ping` / `poll` / `push`)

25.1 Contexto técnico

CIBA define tres formas en que el AS comunica el resultado de la autorización al cliente:

Modo	Cómo
<code>poll</code>	El cliente hace POST repetidos a <code>/token</code> con <code>auth_req_id</code> hasta recibir éxito o <code>expired_token</code> .
<code>ping</code>	El AS hace POST al <code>backchannel_client_notification_endpoint</code> del cliente avisando "lista la respuesta"; el cliente entonces llama a <code>/token</code> .
<code>push</code>	El AS hace POST al <code>backchannel_client_notification_endpoint</code> con los tokens directamente dentro del cuerpo (JWT firmado).

Diagrama comparativo

```

poll:  AS  ← /token (varios polls)  → Cliente
ping:  AS  → POST /notify (sin tokens) → Cliente
       AS  ← /token (una sola vez)  → Cliente
push:  AS  → POST /notify (con tokens) → Cliente  ⚠ NO usado en FAPI
  
```

Por qué `push` casi no se admite en Open Finance

- El AS entrega tokens **sin** que el cliente haga `/token` $\Rightarrow$ no hay mTLS bidireccional en el momento exacto del token-binding.
- El client notification endpoint del cliente recibe tokens; un compromiso de ese endpoint expone tokens.
- Requiere que el cliente exponga un endpoint HTTPS con autenticación fuerte hacia el AS.

25.2 ¿Obligatorio?

- **FAPI-CIBA**: prohíbe `push`. Sólo `ping` o `poll`.
- **SFA Chile**: alinea con FAPI-CIBA: `ping` o `poll`. `ping` preferido por eficiencia.

25.3 Soporte Keycloak $\geq 26.x$

✓ Parcial.

- `poll`: soporte completo.
- `ping`: soporte completo desde 22.x.
- `push`: soporte limitado / experimental; en FAPI-CIBA profile queda deshabilitado.

Clients → CIBA → Backchannel Token Delivery Mode: ping
→ Backchannel Authentication Request Signing Alg: PS256

El AS envía notificación firmada al endpoint del cliente:

```
POST /ciba-cb HTTP/1.1
Host: tpp.example.cl
Content-Type: application/json
Authorization: Bearer <client_notification_token>

{ "auth_req_id": "01H..." }
```

25.4 Recomendaciones

- Forzar ping para mejor UX (no consume polls).
- Endpoint de notificación del cliente protegido por mTLS server-side (cert del AS) y validación del client_notification_token (Bearer firmado).
- Prohibir push por Client Policy.
- Reintentos exponenciales del ping con backoff (max-retry-attempts=3).

26. ACR / AMR — niveles de autenticación

26.1 Contexto técnico

- **ACR (Authentication Context Class Reference)**: representa el *nivel* de seguridad de la autenticación realizada. Es un identificador opaco (URI o número).
- **AMR (Authentication Methods References)**: lista de *métodos* concretos usados (pwd , mfa , otp , hwk , face , fpt , swk).

El cliente solicita un nivel mínimo con acr_values=... en el request. El AS lo cumple (eligiendo el flow apropiado) y lo refleja en el id_token :

```
{
  "acr": "urn:openid:loa:3",
  "amr": ["pwd", "hwk"],
  "auth_time": 1717000000,
  ...
}
```

Niveles típicos en Open Finance

ACR (ejemplo)	Métodos	Caso de uso
urn:loa:1	password	sólo consulta de datos no sensibles
urn:loa:2	password + OTP SMS	consulta balances
urn:loa:3	password + WebAuthn	consulta transacciones detalladas
urn:openbanking:psd2:sca	2 de 3 factores SCA	iniciación de pago

26.2 ¿Obligatorio?

Sí. FAPI 2.0 + SFA Chile exigen Strong Customer Authentication (SCA) basada en al menos 2 factores independientes (conocimiento, posesión, inherencia). Debe ser expresable y verificable vía `acr` / `amr`.

26.3 Soporte Keycloak ≥ 26.x

✓ Soportado.

- **Mapping ACR** → **LoA**: Realm Settings → Authentication → "ACR to Authentication Flow LoA Map".
- **Authentication Flows** con *conditional executors*: ej. "si `acr=urn:loa:3` ⇒ exigir WebAuthn".
- `auth_time`: emitido si `max_age` solicitado por cliente o si flow lo configura.

```
Authentication → Flows → "Browser SFA":
- Cookie
- Username Form
- Conditional - User Configured:
  - WebAuthn Authenticator (requirement: ALTERNATIVE/REQUIRED)
- Conditional - acr level 3:
  - WebAuthn Authenticator (requirement: REQUIRED)
```

26.4 Recomendaciones

- Definir y publicar el catálogo canónico de `acr` (sección 37 para el URN específico del SFA).
- Rechazar tokens con `acr` ausente o por debajo del mínimo del scope/recurso (ej. `payments:write` ⇒ `acr=urn:openbanking:psd2:sca`).
- Auditar `auth_time`: si > 5min antes de un pago, forzar re-autenticación (`max_age=300`).

27. WebAuthn / FIDO2 para SCA

27.1 Contexto técnico

WebAuthn (W3C) + CTAP2 (FIDO Alliance) = **FIDO2**. Permite autenticación con **claves criptográficas** (asimétricas) custodiadas en hardware (TPM, Secure Enclave, YubiKey, llaves NFC) o platform authenticators (Touch ID, Windows Hello). **Phishing-resistente** por diseño (la clave se vincula al *origin* del RP).

Flujo

```
Registro:
RP (banco) → challenge → Authenticator (genera keypair vinculado al origin)
RP ← attestation + public key —
RP guarda credential_id, public_key, sign_count

Login:
RP → challenge → Authenticator (firma con private key)
RP ← assertion (firma + auth_data) —
RP valida firma con la public key guardada
```

Comparado con OTP/SMS:

Aspecto	OTP-SMS	WebAuthn
Phishing-resistant	✗	✓ (vinculado al origen)
MITM-resistant	✗	✓
SIM-swap-resistant	✗	✓
Coste por uso	\$ (SMS)	0
UX	ingresa 6 dígitos	toca/biometría

27.2 ¿Obligatorio?

- **FAPI 2.0**: no exige WebAuthn explícitamente, pero la SCA "phishing-resistant" sólo se cumple razonablemente con FIDO2.
- **SFA Chile**: fuertemente recomendado; muchos bancos lo adoptan como factor obligatorio para SCA en pagos.

27.3 Soporte Keycloak ≥ 26.x

✓ Soportado nativamente.

- Authenticators: WebAuthn Authenticator , WebAuthn Passwordless Authenticator .
- Required Action: Webauthn Register / Webauthn Register Passwordless .
- Soporte para **passkeys** (resident credentials) desde 24.x.

Configuración:

```
Authentication → WebAuthn Policy:
Relying Party Entity Name : Banco Ejemplo
Relying Party ID          : as.banco.cl
Signature Algorithms      : ES256, RS256
Attestation Conveyance Preference : direct      (para AAGUID allow-list)
Authenticator Attachment : cross-platform | platform
Require Resident Key      : Yes (passkeys)
User Verification         : required            (biometría/PIN)
```

27.4 Recomendaciones

- Attestation = direct + AAGUID **allow-list** de autenticadores FIPS 140-2 / FIDO L2 certificados (sólo aceptar marcas/modelos validados).
- User Verification = required (forzar biometría/PIN — no sólo presencia).
- Habilitar passkeys (Resident Key = required) para UX *passwordless* completa.
- Backup de credenciales: ofrecer al usuario registrar **mínimo 2** autenticadores (evita lockout por pérdida).
- Cuando el cliente solicite `acr=urn:openbanking:psd2:sca` , el flow debe forzar WebAuthn (no aceptar password+SMS).

28. RP-Initiated Logout (id_token_hint)

28.1 Contexto técnico

OpenID Connect RP-Initiated Logout 1.0: el cliente (Relying Party) puede solicitar al AS que cierre la sesión del usuario y opcionalmente redirija a una URL del cliente. Endpoint: `end_session_endpoint` .

Parámetros

Param	Obligatorio	Descripción
id_token_hint	sí (FAPI)	id_token previo, prueba que el cliente conocía la sesión
client_id	recomendado	redundante con id_token_hint , ayuda al logging
post_logout_redirect_uri	opcional	URL pre-registrada para redirigir tras logout
state	opcional	echo
logout_hint	opcional	hint para acelerar identificación de sesión
ui_locales	opcional	idioma de la pantalla de confirmación

Diagrama



28.2 ¿Obligatorio?

Sí para FAPI 2.0 y SFA Chile: el usuario debe poder cerrar sesión y la propagación del logout debe ser auditable.

28.3 Soporte Keycloak ≥ 26.x

✔ Soportado nativamente desde 18.x (implementación completa OIDC RP-Initiated Logout 1.0).

Comportamiento por defecto:

- Si `id_token_hint` ausente ⇒ Keycloak muestra una pantalla de **confirmación** al usuario (configurable).
- Validación de `post_logout_redirect_uri` contra whitelist por cliente: `Clients` → `<cliente>` → `Valid post logout redirect URIs` .

```
end_session_endpoint:
  https://as.banco.cl/realms/sfa/protocol/openid-connect/logout
```

28.4 Recomendaciones

- Hacer obligatorio `id_token_hint` (evita ataques CSRF de logout). Configurable vía *Client Policy*:

```
client-attributes:
  id.token.hint.required: "true"
```

- Whitelist de `post_logout_redirect_uri` **exact-match** (sin wildcards). Sección 40.
- Habilitar `Front-Channel Logout` y `Back-Channel Logout` (sección 29) para propagar a otros RPs.
- Auditar evento `LOGOUT` con `client_id`, `user_id`, `sid`.

29. Back-Channel Logout (OIDC BCL 1.0)

29.1 Contexto técnico

OpenID Connect Back-Channel Logout 1.0: cuando una sesión SSO termina (por logout explícito, expiración o revocación de consentimiento), el AS envía un **HTTP POST firmado** (un *Logout Token*) a cada RP que tuvo sesión activa.

Diferente del *Front-Channel Logout* (iframes en el navegador), el back-channel funciona aunque el navegador del usuario esté cerrado.

Logout Token (JWT)

```
{
  "iss": "https://as.banco.cl/realms/sfa",
  "aud": "client-123",
  "iat": 1717000000,
  "jti": "01H...",
  "sub": "248289761001",
  "sid": "08a5019c-17e1-...-...",
  "events": {
    "http://schemas.openid.net/event/backchannel-logout": {}
  }
}
```

El cliente valida firma (`iss` / `aud` / `iat` / firma con JWKS del AS), comprueba `events`, y termina la sesión local del usuario / invalida tokens.

Diagrama

Trigger: usuario hace logout en **RP-A**

```
RP-A → /end_session AS → {
  → POST /bcl con logout_token → RP-B
  → POST /bcl con logout_token → RP-C
  → POST /bcl con logout_token → RP-D
}
```

(en paralelo, mTLS server-side a cada **RP**)

29.2 ¿Obligatorio?

Sí en SFA Chile para propagación de revocación de consentimiento: cuando el usuario revoca un consentimiento en su banco, todos los TPPs que sostenían sesión deben enterarse.

29.3 Soporte Keycloak ≥ 26.x

✔ Soportado nativamente.

```
Clients → <cliente> → Settings:
  Backchannel Logout URL           : https://tpp.example.cl/bcl
  Backchannel Logout Session Required : ON
  Backchannel Logout Revoke Offline Sessions : ON
```

Algoritmo del logout_token: hereda el de id_token (PS256/ES256 en FAPI).

29.4 Recomendaciones

- Endpoint del cliente protegido por mTLS server-side (cert del AS).
- Cliente valida en orden: firma, iss, aud, iat reciente (< 2 min), jti no visto antes (cache), events contiene el URI esperado, sub y/o sid presentes.
- Reintentos: el AS reintenta con backoff exponencial 3 veces; idempotencia garantizada por jti.
- Auditar fallos: si un BCL falla 3 veces, alertar al SIEM (posible RP comprometido o caído).

30. Algoritmos JWE para claims sensibles

30.1 Contexto técnico

JWE (JSON Web Encryption, RFC 7516) cifra el payload de un JWT. En FAPI/Open Finance se usa para:

- **id_token cifrado**: cuando contiene PII (nombre, RUT, dirección).
- **request_object cifrado**: cuando contiene login_hint con CI/RUT.
- **JARM cifrado**: cuando la respuesta lleva información sensible.

Algoritmos recomendados

Componente	alg (key management)	enc (content encryption)
Recomendado	ECDH-ES+A256KW	A256GCM
Alternativa	RSA-0AEP-256	A256GCM
Prohibidos	RSA1_5, dir con clave estática	A128CBC-HS256 (sin PFS)

ECDH-ES provee **Perfect Forward Secrecy** (genera una clave efímera por cada cifrado).

Estructura

JWE compact: <header>.<encrypted_key>.<iv>.<plaintext>.<tag>

Header: { "alg":"ECDH-ES+A256KW", "enc":"A256GCM", "kid":"client-enc-1", "epk":{"..."} }

30.2 ¿Obligatorio?

- **FAPI 2.0:** opcional.
- **SFA Chile:** recomendado para claims con PII. Algunas guías lo exigen para `id_token` cuando incluye RUT.

30.3 Soporte Keycloak ≥ 26.x

✓ Soportado.

Clients → <cliente> → Advanced → Fine Grain OpenID Connect Configuration:

```
ID Token Encrypted Response Alg : ECDH-ES+A256KW
ID Token Encrypted Response Enc : A256GCM
Request Object Encryption Alg   : ECDH-ES+A256KW
Request Object Encryption Enc   : A256GCM
User Info Encrypted Response Alg/Enc : (idem)
Authorization Encrypted Response Alg/Enc : (JARM cifrado)
```

La clave pública de cifrado del cliente debe estar en su `jwks_uri` con `use:"enc"`.

30.4 Recomendaciones

- Limitar JWE a casos con PII; cifrar todo es overhead innecesario.
- Preferir `ECDH-ES` (PFS) sobre `RSA-OAEP`.
- Rotar claves de cifrado del cliente trimestralmente.
- No mezclar firma y cifrado en un solo JWT: usar **Nested JWT** ({ JWE(JWS(payload)) }).

31. Cripto-agilidad y *post-quantum readiness*

31.1 Contexto técnico

Cripto-agilidad = la capacidad del sistema de cambiar algoritmos criptográficos **sin reescribir código ni interrumpir el servicio**. Crítico para:

- Migrar de SHA-1 → SHA-256 → SHA-384 conforme NIST deprecate.
- Adoptar **Post-Quantum Cryptography (PQC)** cuando NIST estandarice (ya hay finalistas: **ML-DSA (Dilithium)** para firmas, **ML-KEM (Kyber)** para intercambio de claves).

Principios

1. **No hardcodear algoritmos** — declararlos en configuración o por `kid` en JWKS.
2. **Soportar múltiples algoritmos simultáneamente** (durante migración).
3. `alg` **dinámico** vía JWS header en cada token.

4. **Hybrid signatures** (transición): firmar con clásico y PQC simultáneamente.

Timeline esperado

2024 NIST FIPS 203/204/205 finalizados (ML-KEM, ML-DSA, SLH-DSA)
2026 Browsers/CAs adoptan hybrid (X25519+ML-KEM) en TLS 1.3
2030 Recomendación NIST: deprecarse RSA-2048/ECC-P256 para datos a largo plazo
2035 Posible obligación regulatoria de PQC en finanzas

31.2 ¿Obligatorio?

- **FAPI 2.0**: aún no exige PQC.
- **SFA Chile**: en hoja de ruta. CMF probablemente alineará con NIST hacia 2027-2028.

31.3 Soporte Keycloak ≥ 26.x

⚠ Parcial.

- Algoritmos clásicos (PS256 , ES256 , EdDSA Ed25519) ✓.
- **ML-DSA / ML-KEM**: aún no nativos. Requieren BouncyCastle PQC provider o JDK 24+ con módulos PQC.
- *Hybrid TLS* (X25519 + ML-KEM): depende de la JVM + Quarkus/Netty stack.

31.4 Recomendaciones

- **No hardcodear** algoritmos en código de SPIs custom; leer del JWKS por kid .
- Prototipar con **BouncyCastle PQC** un SignatureProvider experimental para ML-DSA.
- Mantener jwks_uri con múltiples kid / alg durante migraciones (overlap mínimo 6 meses).
- Inventario centralizado de algoritmos usados (Realm, Clients, eventos firmados, sello de logs).
- Documentar un **Plan de Migración Cripto** (anexo gobernanza).

32. Rate limiting / anti-bruteforce en endpoints OAuth

32.1 Contexto técnico

Endpoints OAuth (/token , /par , /register , /revoke , /userinfo) son blanco de:

- **Brute force de client_secret** (mitigado al prohibirlos, pero subsiste contra code).
- **Code-stuffing**: ataque que prueba millones de code aleatorios.
- **PAR flooding**: agotar memoria con request_uri falsos.
- **DCR flooding**: agotar el AS con registros bogus.
- **Token introspection abuse**: enumerar tokens.

Estrategia

Endpoint	Límite típico
/par	100/min por <code>client_id</code> , 10/s por IP
/token	60/min por <code>client_id</code>
/register (DCR)	10/h por IP, 1 cada 60s por <code>software_id</code>
/userinfo	1000/min por token
/revoke , /introspect	100/min por <code>client_id</code>
Login user	5 fallos en 5 min ⇒ lock 15 min

Respuestas: HTTP 429 Too Many Requests + Retry-After .

32.2 ¿Obligatorio?

Sí. SFA Chile (resiliencia + protección anti-fraude). FAPI 2.0 lo trata como *operational concern* pero la conformance recomienda probarlo.

32.3 Soporte Keycloak ≥ 26.x

⚠ Parcial.

- **Brute Force Detection** para login de usuario: ☒ nativo (Realm Settings → Security Defenses → Brute Force Detection).
- **Rate limiting en endpoints OAuth** por `client_id` /IP: ☒ no nativo. Requiere API Gateway externo.

32.4 Recomendaciones

- Desplegar gateway (**Kong, Apigee, Envoy/Istio, AWS API Gateway**) delante de Keycloak con políticas:

```
plugins:
- name: rate-limiting
  config:
    minute: 100
    policy: redis           # cluster-wide
    identifier: consumer   # por client_id
    hide_client_headers: false
```

- Métrica Prometheus de 429s; alerta si tasa > 5% de requests legítimos.
- Listas negras dinámicas: si un `client_id` excede su cuota 3 veces en 1h, bloquearlo 24h e investigar.
- Para DCR, además del rate limit usar **CAPTCHA** o **invitation token** (`initial_access_token`).

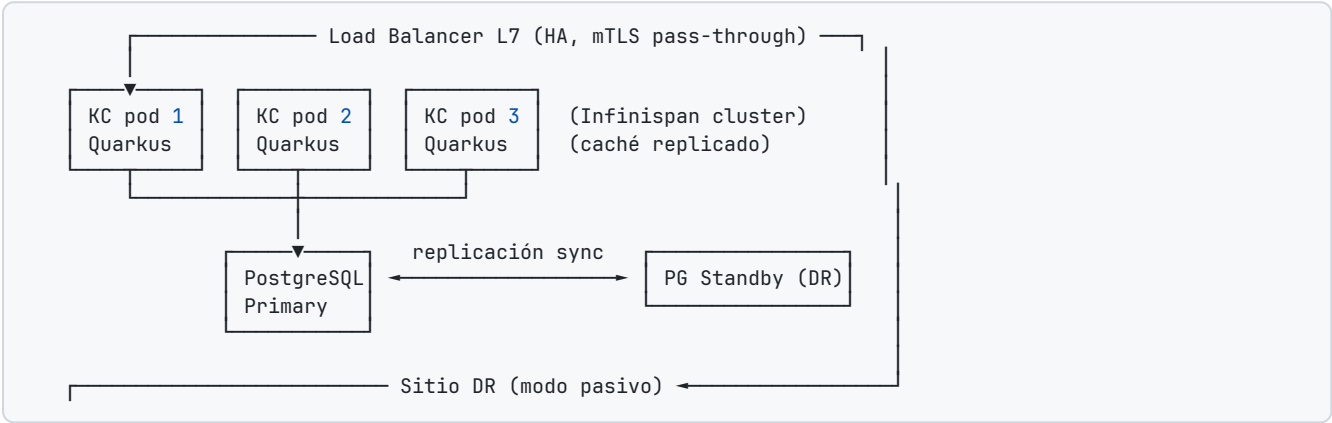
33. Resiliencia / HA del Authorization Server

33.1 Contexto técnico

Por ser servicio crítico, el AS necesita:

- **RTO** (Recovery Time Objective) ≤ 30 min (típico SFA).
- **RPO** (Recovery Point Objective) ≤ 5 min.
- **SLA** $\geq 99.9\%$ mensual (CMF puede exigir 99.95%).
- **Disaster Recovery** activo-pasivo o activo-activo en sitio secundario.

Arquitectura HA típica



Componentes a considerar

Componente	Replicación
BD (sessions, clients, users)	PostgreSQL streaming replication
Infinispan cache (sessions, action tokens)	distributed cache owners=2 , cross-site con XSite
JWKS keys (firma)	HSM con sync entre sites
Logs/auditoría	Kafka MirrorMaker 2
Config (realms)	GitOps (Realm export → repo → import)

33.2 ¿Obligatorio?

Sí. SFA Chile define SLAs estrictos para los AS de instituciones financieras.

33.3 Soporte Keycloak $\geq 26.x$

✓ Soportado.

- **Quarkus + Infinispan**: cluster nativo (TCP/UDP/Kubernetes discovery).
- **PostgreSQL HA**: vía Patroni, Stolon, AWS RDS Multi-AZ, etc.
- **XSite replication** (cross-site Infinispan): documentado por Red Hat.
- **Active-Active multi-site**: documentado en *Keycloak HA Multi-Site Guide* (26.x).
- **Kubernetes Operator** oficial: keycloak.org/v2alpha1 (Helm + Operator).

33.4 Recomendaciones

- Mínimo **3 pods** Keycloak por sitio (quorum Infinispan).
- BD: read replica adicional para queries de auditoría / introspección masiva.
- **DR drills trimestrales**: failover manual end-to-end.
- **Chaos engineering**: matar un pod, una zona, el sitio entero; medir RTO real.
- **Health probes**: `/health/live` + `/health/ready` + `/metrics` (Quarkus MicroProfile).
- **Capacity planning**: 1 vCPU = ~200 logins/s; dimensionar para pico × 2.

34. Time synchronization (NTP / NTS)

34.1 Contexto técnico

Toda la validación de tokens depende del reloj: `iat`, `exp`, `nbf`, `auth_time`. Un *drift* de pocos minutos rompe validaciones legítimas y abre ventanas de replay.

NTP (Network Time Protocol, RFC 5905) es el estándar, pero **no autenticado** por defecto ⇒ vulnerable a MITM que adelanten/atrasen el reloj para forzar aceptación de tokens expirados o de futuro.

NTS (Network Time Security, RFC 8915) añade autenticación criptográfica al NTP usando TLS para el setup y AEAD para los paquetes.

Diagrama



Métricas de salud

- **Offset** del reloj vs upstream: < 50ms (alerta > 100ms).
- **Stratum**: idealmente ≤ 3.
- **Last sync**: < 5 min.

34.2 ¿Obligatorio?

Implícito. SFA Chile lo asume; sin NTP funcional, la conformance suite falla ampliamente.

34.3 Soporte Keycloak ≥ 26.x

Depende del sistema operativo (Keycloak usa `System.currentTimeMillis()`).

- Si el SO tiene `chrony` o `systemd-timesyncd` configurado.
- NTS requiere `chrony` ≥ 4.0 con `nts` `enabled` o `ntpsec`.

34.4 Recomendaciones

- Usar **NTS** (autenticado) en producción:

```
# /etc/chrony/chrony.conf
server time.cloudflare.com iburst nts
server time.nist.gov iburst nts
```

- Mínimo **2 fuentes diversas** (geográfica y administrativamente).
- Monitor Prometheus de offset (`node_timex_offset_seconds`), alerta > 100ms.
- En cluster, todos los pods deben sincronizar contra la misma fuente.
- Documentar política de leeway en tokens (`exp +5s`, `iat -10s`) y mantener consistente.

35. Inventario de eventos y política de retención (5-10 años)

35.1 Contexto técnico

SFA Chile + Ley 19.628 (Protección de Datos Personales) + normativa CMF exigen retención de:

- **Logs de autenticación:** login OK/fail, MFA usado, IP, dispositivo.
- **Eventos de consentimiento:** creación, modificación, revocación, expiración.
- **Logs criptográficos** (sección 18): validación de certs, validación de SSAs.
- **Cambios de configuración** del AS (admin events).
- **Token-issuance** y **token-revocation**.
- **Logs de errores** (failed authorization, invalid_grant, etc.).

Retención mínima: **5 años**. En algunos casos (pagos > UF 500, fraude investigado): **10 años**.

Catálogo de eventos a definir

Categoría	Eventos	Retención
Authentication	LOGIN, LOGIN_ERROR, LOGOUT, IDENTITY_PROVIDER_LOGIN	5 años
Token	TOKEN_ISSUED, REFRESH_TOKEN, TOKEN_REVOKED	5 años
Consent	CONSENT_GRANTED, CONSENT_UPDATED, CONSENT_REVOKED	10 años
Client	CLIENT_REGISTER, CLIENT_UPDATE, CLIENT_DELETE	10 años
Admin	REALM_UPDATE, ROLE_*, AUTHENTICATOR_*	10 años
Crypto (sec 18)	MTLS_HANDSHAKE, OCSP_CHECK, SSA_VALIDATED	5 años

35.2 ¿Obligatorio?

Sí. CMF + Ley 19.628 (reforma 2024).

35.3 Soporte Keycloak ≥ 26.x

⚠ Parcial.

- **Login Events** y **Admin Events** ✓ nativos, configurables en `Realm Settings → Events`.
- Retención en BD: configurable pero pensada para corto plazo (días/semanas).
- Almacenamiento a largo plazo: **no nativo**. Requiere `EventListenerProvider` SPI que exporte a sistema externo (Elasticsearch, Kafka → S3, etc.).

```
Realm Settings → Events:
Save Events           : ON
Expiration            : 30 days (corto plazo en BD)
Events Listeners      : jboss-logging, custom-kafka-publisher
```

35.4 Recomendaciones

- Implementar `EventListenerProvider` que envía a **Kafka** topic `audit.raw`.
- Pipeline downstream: Kafka → Flink/Logstash → **S3 con Object Lock (compliance mode)** + Elasticsearch (consulta).
- Cifrar at-rest (SSE-KMS).
- Inmutabilidad: WORM storage o append-only ledger.
- Documentar el **catálogo de eventos** y su mapeo a artículos normativos.
- Pruebas anuales de **recuperación**: extraer eventos de 4 años atrás y verificar integridad.

36. Conformance OIDF + Pen-testing recurrentes

36.1 Contexto técnico

OpenID Foundation Conformance Suite: software open-source que ejecuta cientos de pruebas automatizadas contra un AS para verificar conformidad con perfiles FAPI 1.0, FAPI 2.0, OIDC, CIBA, etc. Disponible en <https://openid.net/certification/>.

Cubre:

- Algoritmos firma/cifrado válidos.
- Comportamiento ante parámetros inválidos.
- Rechazo de flujos prohibidos.
- Validación de PAR, JAR, JARM, DPoP, mTLS.
- Issuer identification (RFC 9207).
- Manejo de errores.

Para certificarse formalmente, el AS debe:

1. Ejecutar la suite y obtener PASS en todos los tests del plan elegido (`fapi2-security-profile-id2-test-plan`).
2. Subir el log de resultados a OIDF.
3. Pagar fee de certificación.
4. Recibir badge oficial.

Pen-testing: pruebas manuales/semi-automatizadas por terceros independientes, foco en OAuth attacks (mix-up, code injection, IDP confusion, CSRF, redirect manipulation).

36.2 ¿Obligatorio?

- **FAPI 2.0:** el badge OIDF no es estrictamente obligatorio por la norma, pero es la **única forma reconocida** de demostrar conformidad.
- **SFA Chile:** la CMF puede exigir certificación inicial y **recertificación anual**.
- **Pen-testing:** anual por norma sectorial financiera chilena (RAN 20-7 de la CMF para ciberseguridad).

36.3 Soporte Keycloak ≥ 26.x

N/A (es proceso, no producto). Sin embargo:

- La comunidad Keycloak mantiene configuraciones de referencia que pasan la suite.
- Red Hat Single Sign-On (build comercial) provee runbooks de conformance.

36.4 Recomendaciones

- Integrar la conformance suite en CI: por cada release de Keycloak (incluyendo upgrades), correr el plan.
- Mantener un **realm de pruebas** dedicado (`realm-fapi-conformance`) con configuración congelada.
- Pen-testing anual con vendor especializado en OAuth/OIDC (no genérico web).
- Documentar y remediar findings; mantener trazabilidad.
- Coordinar con CMF la entrega del *Conformance Report*.

```
docker run --rm \
-v $PWD/sfa-conformance.json:/conf \
openidfoundation/conformance-suite \
--test-plan fapi2-security-profile-id2-test-plan \
--variant sender_constrain=mtls,client_auth_type=mtls,fapi_response_mode=jarm
```

37. Mapeo de `acr` canónico SFA (`urn:cmf:cl:sfa:...`)

37.1 Contexto técnico

Distinto de la sección 26 (concepto general de ACR/AMR), aquí se trata el **valor canónico específico** que el SFA Chile probablemente publicará (o ya publicó) para denotar SCA conforme a la NCG 514.

Ejemplo previsible:

```
urn:cmf:cl:sfa:sca:level-3
urn:cmf:cl:sfa:auth:psd2-compliant
urn:cmf:cl:sfa:auth:read-only
urn:cmf:cl:sfa:auth:payment-init
```

El AS debe:

- Aceptar estos valores en `acr_values`.
- Mapearlos a flujos de autenticación con métodos adecuados.
- Reflejar el ACR efectivo en `id_token.acr` (debe ser $\geq$ al solicitado).
- Rechazar el request si el AS no puede cumplir el ACR mínimo.

37.2 ¿Obligatorio?

Sí una vez la CMF publique el URN canónico. Mientras tanto, usar un URN provisorio interno y migrar.

37.3 Soporte Keycloak $\geq$ 26.x

✓ Soportado.

Realm Settings → Authentication → "ACR to Authentication LoA Map":

```
urn:cmf:cl:sfa:sca:level-3      → 3
urn:cmf:cl:sfa:auth:read-only   → 1
urn:cmf:cl:sfa:auth:payment-init → 3
```

Authentication Flow "Browser SFA" tiene Conditional executors por LoA.

Y *Protocol Mapper* del `id_token` que normaliza el output de `acr`.

37.4 Recomendaciones

- Suscribirse a publicaciones CMF para captar el URN definitivo apenas se libere.
- Documentar el mapeo `URN ↔ LoA ↔ métodos` en una tabla versionada en el repo.
- Validar con la conformance suite que el AS emite el ACR exacto solicitado (no uno mayor o menor sin justificación).
- Si el cliente solicita un ACR no soportado: rechazar con `error=invalid_request` y `error_description` claro.

38. Notificación de eventos al cliente (SET / RFC 8417)

38.1 Contexto técnico

RFC 8417 — Security Event Token (SET): JWT especial con `Content-Type: application/secevent+jwt`, transportado vía HTTP POST a un webhook del cliente. Estandariza la notificación de eventos asíncronos

entre AS y RP/RS.

Casos de uso en SFA:

- **Revocación de consentimiento** por el usuario.
- **Bloqueo de cuenta** (kyc-failed, logout).
- **Cambio de contraseña** / re-enrolamiento (sesiones a invalidar).
- **Detección de fraude** que requiere invalidar tokens activos.

Estructura de SET

```
Header: { "alg": "PS256", "kid": "as-key-1", "typ": "secevent+jwt" }
Payload:
{
  "iss": "https://as.banco.cl/realms/sfa",
  "aud": "client-123",
  "iat": 1717000000,
  "jti": "01H...",
  "events": {
    "https://schemas.openbanking.cl/event/consent-revoked": {
      "consent_id": "ctx-9d8e...",
      "user_id": "248289761001",
      "reason": "user-revoke"
    }
  },
  "sub": "248289761001"
}
```

Estándares relacionados:

- **OpenID Shared Signals Framework (SSF/CAEP)**: protocolo de descubrimiento + suscripción.
- **RISC (Risk and Incident Sharing and Coordination)** para fraude.

Diagrama

```
Usuario en banco app
├── revoca consent
│   ├── AS publica evento en Kafka topic "sfa.events"
│   │   ├── SET dispatcher service
│   │   │   ├── POST /event con SET firmado → endpoint del TPP (mTLS)
│   │   │   └── TPP invalida tokens y notifica al usuario
```

38.2 ¿Obligatorio?

Sí en SFA Chile para notificación de revocación de consentimiento (eco a Open Finance Brasil que ya lo formaliza).

38.3 Soporte Keycloak ≥ 26.x

✗ No nativo.

Keycloak emite eventos internamente (`EventListenerProvider`), pero no construye, firma y despacha SETs hacia clientes externos. Requiere:

- SPI `EventListenerProvider` custom que capture eventos relevantes.

- Servicio dispatcher externo (Quarkus/Spring) que firma con HSM y envía mTLS POST.

38.4 Recomendaciones

- Construir `sfa-event-dispatcher` : microservicio en Quarkus que:
 - Consume Kafka `sfa.events` .
 - Por cada evento, busca clientes suscritos (vía Consent API o registry).
 - Firma SET (PS256, HSM-backed).
 - HTTP POST con `Content-Type: application/secevent+jwt` al endpoint del cliente.
 - Reintentos exponenciales (3 intentos, backoff 1m/5m/30m) y DLQ.
 - Idempotencia: `jti` único persistido en lado cliente.
 - Endpoint del cliente debe responder `202 Accepted` rápido (procesamiento asíncrono).
 - Catálogo público de tipos de evento (URI namespace `https://schemas.openbanking.cl/event/*`).
 - Considerar adopción futura del **SSF/CAEP**.
-

39. `sub` pseudonymous / pairwise / no-PII

39.1 Contexto técnico

El claim `sub` (subject) identifica al usuario dentro del `id_token` y `access_token` . Por privacidad:

- **NO** debe ser PII directa (RUT, email, número de cliente real).
- Debe ser **pseudonymous** (id sintético, irreversible sin acceso al AS).
- Debe ser **pairwise**: distinto **por cliente** para evitar correlación entre TPPs (mismo usuario en TPP-A y TPP-B no debe ser identificable como la misma persona).

OIDC define dos tipos:

- **public** : mismo `sub` para todos los clientes (no recomendado en Open Finance).
- **pairwise** : derivado de `(sector_identifier, user_id, salt)` ⇒ único por sector.

Algoritmo pairwise (recomendado OIDC §8.1)

```
sub_pairwise = base64url( SHA256( sector_identifier || user_id || salt ) )
```

donde `sector_identifier` viene del cliente (URI o agrupación de clientes del mismo sector).

39.2 ¿Obligatorio?

Sí. SFA Chile + Ley 19.628 (minimización de datos). FAPI 2.0 no lo exige explícitamente pero la conformance suite recomienda `pairwise` .

39.3 Soporte Keycloak ≥ 26.x

✓ Soportado.

- Por defecto, `sub` = UUID interno del usuario (no PII, pero **público** — mismo para todos los clientes).
- **Pairwise subject identifier**: vía *Protocol Mapper* `pairwise-sub-mapper`.

```
Clients → <cliente> → Client Scopes → openid → Mappers
+ Add Mapper "pairwise subject identifier"
  Sector Identifier URI: https://tpp.example.cl/sector
  Salt:                  <secreto-largo-rotable>
```

39.4 Recomendaciones

- Usar **pairwise** para todos los clientes Open Finance.
- `sector_identifier_uri` agrupa redirect URIs autorizadas (RFC OIDC); el TPP publica un JSON.
- **NO** rotar el salt sin migración (cambiaría el `sub` y rompería sesiones de larga duración).
- Para migración desde `sub=email`: emitir un *protocol mapper* transicional que devuelva tanto el viejo como el nuevo `sub` durante un periodo de gracia, luego cambiar.
- Auditar que ningún `sub` emitido contenga `@`, `-` típicos de RUT, etc.

40. Exact-match de `redirect_uri`

40.1 Contexto técnico

FAPI 2.0 prohíbe matching por:

- **Wildcards** (`https://tpp.example.cl/*`).
- **Prefijos** (`https://tpp.example.cl/cb*`).
- **Query/fragment laxo**.

Sólo se admite comparación **byte-exact** entre el `redirect_uri` del request y los URIs pre-registrados. Cualquier discrepancia ⇒ rechazo con `error=invalid_request`.

Vector que mitiga

```
Cliente registrado con: https://tpp.example.cl/*
Atacante crea redirect: https://tpp.example.cl/anywhere?victim=true
                        → AS valida prefix-match → OK
                        → entrega code al sub-recurso controlado por atacante
```

40.2 ¿Obligatorio?

Sí. FAPI 2.0 + RFC 8252 *OAuth 2.0 for Native Apps* + best practices BCP 240.

40.3 Soporte Keycloak ≥ 26.x

✓ Soportado.

- Keycloak compara `Valid Redirect URIs` con el `redirect_uri` del request usando *exact-match* **siempre que no haya wildcards** en la configuración.
- ⚠️ **La UI permite wildcards** (`*` y `+`). Esto es peligroso para FAPI.

Bloquear con *Client Policy*:

```
secure-redirect-uris-enforcer-executor:
  - rechaza redirect_uri con * o + en cualquier client matchado por la policy
```

40.4 Recomendaciones

- Habilitar el executor `secure-redirect-uris-enforcer` para todos los clientes FAPI.
- Auditoría mensual: query a la BD para detectar clientes con `*` en sus redirect URIs:

```
SELECT client_id, value FROM redirect_uris
WHERE value LIKE '%*%' OR value LIKE '%+%';
```

- Para nuevos clientes registrados vía DCR, validar en el `ClientRegistrationPolicy` que ningún `redirect_uri` tenga wildcards.
- Mismo principio para `post_logout_redirect_uri` y `request_uris`.

41. Prohibición de `request_uri` pre-registrado

41.1 Contexto técnico

OIDC original permitía registrar `request_uris` estáticos en el cliente (URIs apuntando a JWTs JAR alojados por el cliente). El AS los descargaba y validaba.

FAPI 2.0 lo **prohíbe** porque:

- Permite ataques **SSRF** (AS forzado a fetch URIs internas).
- Permite **caché poisoning** entre requests.
- No garantiza autenticación del cliente al *publicar* el request.

En su lugar, FAPI 2.0 exige que `request_uri` provenga **siempre** de una respuesta a **PAR** (sección 11): el cliente hace POST mTLS al AS, el AS devuelve un `request_uri` opaco efímero.

Diferencia

```
Pre-registrado (PROHIBIDO):
  Cliente registra request_uris=["https://tpp.example.cl/jar"]
  /authorize?...&request_uri=https://tpp.example.cl/jar
  AS hace GET a esa URL (SSRF risk!)

PAR (REQUERIDO):
  Cliente POST mTLS a /par → AS guarda request → devuelve request_uri=urn:ietf:params:oauth:request_uri:abc
  /authorize?...&request_uri=urn:ietf:params:oauth:request_uri:abc
  AS recupera de su propia memoria (no fetch externo)
```

41.2 ¿Obligatorio?

Sí. FAPI 2.0 Security Profile §5.3.1.2.

41.3 Soporte Keycloak ≥ 26.x

✓ Soportado.

El profile `fapi-2-security-profile` activa el executor `secure-request-object-executor` que:

- Rechaza `request_uri` que no comience con `urn:ietf:params:oauth:request_uri:` (formato PAR de Keycloak).
- Ignora cualquier `Request URIs` estático configurado en el cliente.

41.4 Recomendaciones

- Verificar que el campo `Request URIs` en *Clients* → *Advanced* esté **vacío** para todos los clientes FAPI.
- Limpiar URIs heredadas vía script al migrar de FAPI 1.0 a 2.0.
- Auditar config drift: alerta si algún admin añade `request_uris` estático.
- Test de regresión en CI: enviar request con `request_uri=https:// ...` y verificar rechazo.

42. Consent API formal (CRUD de `consent_id`)

42.1 Contexto técnico

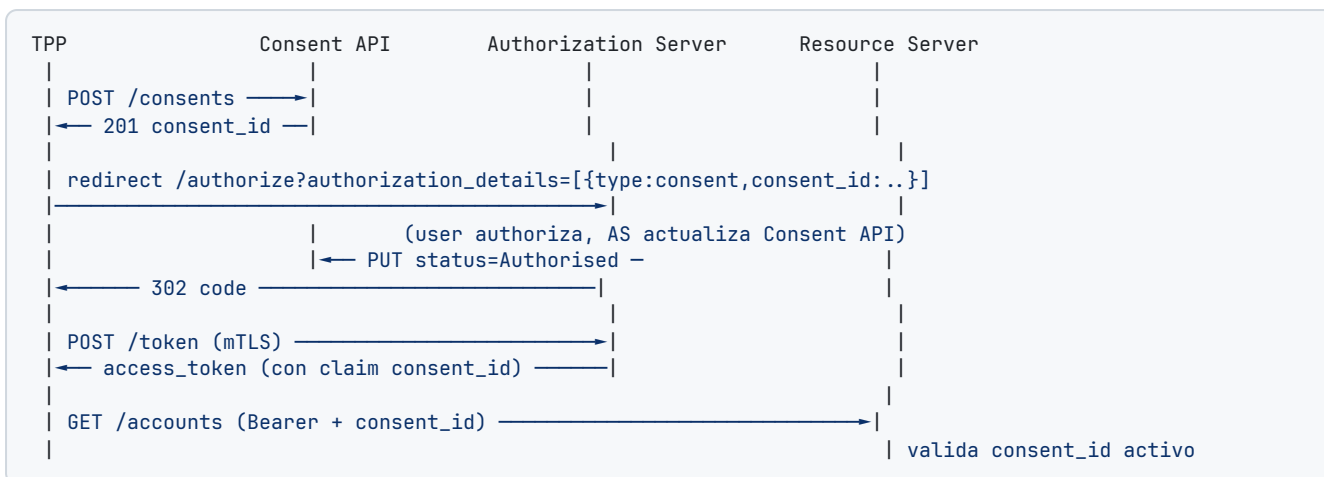
Distinto de la sección 15 (granularidad y UX del consentimiento) y de la sección 16 (RAR como contenedor de detalles), aquí se trata el **endpoint REST formal** que el banco expone para que los TPPs **creen, consulten, modifiquen y revoquen** consentimientos como recurso de primera clase.

Modelo (similar a Open Banking Brasil / UK):

POST	<code>/consents</code>	(TPP crea <code>consent_id</code> , AS responde <code>"AwaitingAuthorisation"</code>)
GET	<code>/consents/{consent_id}</code>	(estado: <code>Authorised</code> <code>Rejected</code> <code>Revoked</code> <code>Expired</code>)
DELETE	<code>/consents/{consent_id}</code>	(revocación por TPP o por usuario)
PATCH	<code>/consents/{consent_id}/extend</code>	(extender vigencia)

El TPP **primero** crea el `consent_id`, luego inicia el flow OAuth pasándolo en `authorization_details` (RAR) o como scope dinámico. El usuario autoriza, el banco actualiza el estado a `Authorised`. Cualquier API call posterior valida el `consent_id` referenciado.

Diagrama



42.2 ¿Obligatorio?

Sí. SFA Chile lo prescribe explícitamente (modelo Open Banking Brasil).

42.3 Soporte Keycloak ≥ 26.x

✗ No es responsabilidad del AS; es un servicio aparte. Keycloak **se integra** con la Consent API pero no la implementa.

42.4 Recomendaciones

- Construir microservicio Consent API:
 - Stack: Quarkus/Spring Boot + PostgreSQL.
 - Auth: mTLS hacia los TPPs (cert del DCR).
 - Auditoría: cada cambio de estado emite evento Kafka (alimenta logs sec 18 y SETs sec 38).
- Esquema mínimo del `consent` :

```

{
  "consent_id": "ctx-...",
  "status": "Authorised",
  "creation_datetime": "2026-05-25T16:00:00Z",
  "expiration_datetime": "2027-05-25T16:00:00Z",
  "permissions": ["ReadBalances", "ReadTransactionsBasic"],
  "user_id": "248289761001",
  "tpp_id": "client-123"
}
  
```

- Integración con Keycloak: *Authenticator* SPI custom que:
 - Recibe `authorization_details` con `consent_id`.
 - Consulta Consent API para obtener detalles y renderizar consent screen enriquecida.
 - Al aprobar, llama a Consent API para cambiar estado a `Authorised`.
 - Inyecta `consent_id` en el `access_token` (ya sea como claim o dentro de `authorization_details`).

43. Aislamiento por *tenant* / realm

43.1 Contexto técnico

Si una sola instalación de Keycloak sirve a múltiples instituciones o filiales, hay que evitar:

- **Cross-tenant token forgery**: un cliente de tenant A consigue un token aceptable por tenant B.
- **Cross-tenant data leak**: queries/admin de un tenant exponen datos de otro.
- **Cross-tenant key compromise**: una clave comprometida afecta varios tenants.

Estrategias en Keycloak

Estrategia	Aislamiento	Costo operativo
Realm por tenant	Issuer, keys, users, clients, themes separados	Bajo (recomendado)
Instancia separada por tenant	Total	Alto
Multi-tenant en mismo realm via groups	⚠ Insuficiente	Bajo, pero no apto para SFA

Recomendación: **un realm por institución financiera.**

Diagrama

```
Keycloak instance
├── realm sfa-banco-a (issuer https://kc/.../realms/sfa-banco-a)
│   ├── keys (HSM partition A)
│   ├── users (BBDD lógica A)
│   └── clients (TPPs autorizados en banco A)
├── realm sfa-banco-b (issuer https://kc/.../realms/sfa-banco-b)
│   ├── keys (HSM partition B)
│   ├── users (BBDD lógica B)
│   └── clients
└── realm sfa-banco-c ...
```

43.2 ¿Obligatorio?

Sí. Cross-issuer = vulnerabilidad fatal en FAPI 2.0 (rompe mix-up protection).

43.3 Soporte Keycloak ≥ 26.x

✓ Nativo.

- Realm-per-tenant es el modelo recomendado por Red Hat / Keycloak community.
- Cada realm tiene sus propias keys, users, clients, themes, flows.

43.4 Recomendaciones

- **Un realm por institución.**
- HSM con **partición lógica por realm** (PKCS#11 slot).
- Schema PostgreSQL único pero con prefijos / triggers de RLS si se exige aislamiento adicional.

- Themes/login pages por realm (branding propio).
- Monitoreo Prometheus etiquetado por realm.
- Auditoría: nunca compartir `client_id` entre realms (mismo string, distintos issuers ⇒ confusión).
- DR plan documentado por realm.

44. Política de claves: separación de roles + HSM

44.1 Contexto técnico

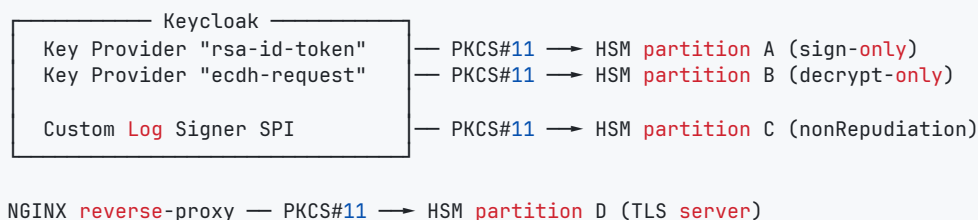
Una sola clave criptográfica usada para múltiples propósitos es mala práctica (compromete *todo* si se filtra).
Política de **separación de roles**:

Rol	Uso	Algoritmo típico	Almacenamiento
<code>id_token-sign</code>	firma <code>id_token</code> , JARM	PS256 / ES256	HSM partition A
<code>request-decrypt</code>	descifra JWEs entrantes (request object)	ECDH-ES	HSM partition B
<code>client-assertion-verify</code>	(clave pública del cliente, en JWKS — no aplica)	—	—
<code>log-sign</code>	firma de logs criptográficos (sec 18)	PS256	HSM partition C, key usage = nonRepudiation
<code>mtls-server</code>	TLS server certificate	RSA-3072 / ECDSA-P256	HSM partition D
<code>ocsp-responder</code>	firma respuestas OCSP (si AS actúa como responder)	RSA-2048	HSM partition E
<code>ssa-verify</code>	(clave pública del Directorio CMF — externa)	—	—

Hardware Security Module (HSM)

- Estándar de comunicación: **PKCS#11**.
- Estándares de cumplimiento: **FIPS 140-2 Level 3** (Thales Luna, Utimaco, AWS CloudHSM, Azure Dedicated HSM, GCP Cloud HSM).
- **Key usage flags**: limitar a operaciones específicas (`CKA_SIGN` , `CKA_DECRYPT` , no `CKA_EXTRACTABLE`).
- **M-of-N quorum** para operaciones de admin (rotación, backup).

Diagrama



44.2 ¿Obligatorio?

Sí. SFA Chile + buenas prácticas NIST SP 800-57. La CMF puede exigir certificación FIPS 140-2 Level 3 para claves de firma de tokens.

44.3 Soporte Keycloak ≥ 26.x

⚠ Parcial.

- **Key Providers nativos:** `rsa-generated`, `ecdsa-generated`, `rsa-enc-generated`, `aes-generated`. Permiten múltiples keys por realm.
- **HSM vía PKCS#11:** configurable a nivel JVM (`security.provider` + `Sun PKCS#11`) o vía extensiones específicas.
- **Key usage por proveedor:** limitado; Keycloak no expone explícitamente flags PKCS#11.

```
# /etc/keycloak/sunpkcs11.cfg
name = LunaHSM
library = /usr/lib/libCryptoki2_64.so
slot = 0

# JVM args
-Djava.security.providers=SunPKCS11=/etc/keycloak/sunpkcs11.cfg
```

Luego en Realm Settings → Keys: añadir Key Provider "java-keystore" apuntando a un keystore PKCS11.

44.4 Recomendaciones

- **HSM en producción**, sin excepciones, para firma de `id_token`, JARM, logs criptográficos.
- **Partición separada por rol** (no compartir keys entre `id_token-sign` y `log-sign`).
- Rotación documentada: **anual** para firma, semestral para cifrado, ante incidente para todas.
- Procedimiento de **rotación con overlap** (publicar nueva key como `passive` 7 días antes de pasarla a `active`).
- **Inventario de claves** versionado (Git) con: rol, alg, kid, hsm-slot, fecha-creación, fecha-rotación-prevista, dueño.
- **Backups cifrados** del material no-extraíble vía mecanismo del HSM (no copia raw).
- **DR:** HSMs replicados cross-site con sincronización de claves (no exportar plaintext).
- **M-of-N quorum** para rotación y borrado.
- Auditoría mensual de cumplimiento.

Apéndice A — Stack de extensión recomendado para Keycloak

```
keycloak-26.x
├── /opt/keycloak/providers/
│   ├── sfa-ssa-validator.jar                (§8, §9)
│   ├── sfa-x509-ocsp-authenticator.jar     (§13)
│   ├── sfa-rar-validator.jar              (§16)
│   ├── sfa-scope-catalog-mapper.jar       (§17)
│   ├── sfa-signed-event-listener.jar      (§18)
│   ├── sfa-consent-bridge-authenticator.jar (§15, §42)
│   ├── sfa-acr-enforcer.jar               (§26, §37)
│   ├── sfa-set-dispatcher-event-listener.jar (§38)
│   ├── sfa-redirect-uri-strict-policy.jar (§40)
│   └── sfa-pairwise-sub-mapper-ext.jar    (§39)
└── /opt/keycloak/conf/keycloak.conf
    features=fapi-2,par,dpop,jarm,ciba,token-exchange
    spi-x509cert-lookup-provider=nginx
    spi-events-listener-jboss-logging-success-level=info
```

Apéndice B — Comandos de validación

Conformance suite (FAPI 2.0 Security Profile + Message Signing):

```
docker run --rm -v $PWD/sfa-conformance.json:/conf \
  openidfoundation/conformance-suite \
  --test-plan fapi2-security-profile-id2-test-plan \
  --variant sender_constrain=mtls,client_auth_type=mtls,fapi_response_mode=jarm

docker run --rm -v $PWD/sfa-conformance.json:/conf \
  openidfoundation/conformance-suite \
  --test-plan fapi2-message-signing-id1-test-plan \
  --variant sender_constrain=mtls,client_auth_type=mtls
```

Verificación TLS / mTLS:

```
openssl s_client -connect as.banco.cl:443 -tls1_3 -status \
  -cert client.pem -key client.key -CAfile cmf-chain.pem
```

Verificación OCSP directa:

```
openssl ocsp -issuer issuer.pem -cert client.pem \
  -url http://ocsp.directorio.cmf.cl -resp_text -respout resp.der
```

Verificación de presencia de `iss` (RFC 9207) en respuesta:

```
curl -is "https://as.banco.cl/realms/sfa/protocol/openid-connect/auth?response_type=code&client_id=x&state=y&" \
  | grep -E '^Location:' | grep -E 'iss='
```

Decodificar un JARM response:

```
echo "<jwt-response>" | cut -d'.' -f2 | base64 -d | jq
```

Apéndice C — Referencias

- OpenID Foundation, **FAPI 2.0 Security Profile**, Final.
- OpenID Foundation, **FAPI 2.0 Message Signing**.

- OpenID Foundation, **FAPI-CIBA**.
- OpenID Foundation, **OIDC RP-Initiated Logout 1.0**.
- OpenID Foundation, **OIDC Back-Channel Logout 1.0**.
- OpenID Foundation, **OIDC Client-Initiated Backchannel Authentication (CIBA)**.
- W3C **WebAuthn Level 2** + FIDO Alliance **CTAP 2.1**.
- RFC 8705 — OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens.
- RFC 9126 — OAuth 2.0 Pushed Authorization Requests.
- RFC 9449 — OAuth 2.0 Demonstrating Proof of Possession (DPoP).
- RFC 9396 — OAuth 2.0 Rich Authorization Requests.
- RFC 9101 — OAuth 2.0 JWT-Secured Authorization Request (JAR).
- RFC 9207 — OAuth 2.0 Authorization Server Issuer Identification.
- RFC 7591 / 7592 — Dynamic Client Registration.
- RFC 7636 — PKCE.
- RFC 7515 / 7516 / 7517 / 7518 / 7519 — JOSE (JWS / JWE / JWK / JWA / JWT).
- RFC 7523 — JWT Profile for OAuth 2.0 Client Authentication.
- RFC 8417 — Security Event Token (SET).
- RFC 8915 — Network Time Security (NTS).
- RFC 6960 — OCSP. RFC 6066 — OCSP Stapling.
- NIST SP 800-57 — Key Management.
- NIST FIPS 203/204/205 — ML-KEM / ML-DSA / SLH-DSA (PQC).
- CMF Chile — Norma de Carácter General N.º 514 y normativa técnica complementaria del SFA.
- CMF Chile — RAN 20-7 (Ciberseguridad).
- Keycloak Server Administration Guide (FAPI section, Client Policies, DPoP, PAR, JARM, CIBA).
- Keycloak Securing Apps and Services Guide.
- Keycloak High-Availability Multi-Site Guide.